

ZETIN Drone

코드베이스 전체 분석서

폴더 · 파일 · 함수 단위 역할과 작동 기제 정리

ESP32-S3 듀얼 IMU 쿼드콥터 비행제어 시스템과 PC 지상국

대상 저장소	Drone_github_DoHyun (branch <code>main</code>)
기준 커밋	<code>b3ec12b</code>
분석 범위	142개 파일 / 소스 약 21,000행
주요 언어	C++ (Arduino), Python 3.12
작성일	2026-09-23

이 문서는 저장소에 실제로 존재하는 코드를 직접 읽어 작성했다. 설계 문서나 계획서의 서술이 아니라 **현재 코드가 실제로 수행하는 동작**을 기준으로 기술하며, 코드와 문서의 서술이 어긋나는 지점은 본문에서 따로 지적한다. 실비행 검증 여부와 코드 구현 여부는 엄격히 구분해 표기했다.

목차

1. 문서 개요와 읽는 법

- 1.1 이 문서의 구성 원칙
- 1.2 표기 규약

2. 시스템 아키텍처 조감

- 2.1 무엇을 만드는 프로젝트인가
- 2.2 하드웨어 구성
- 2.3 소프트웨어 구성 요소와 데이터 흐름
- 2.4 좌표계와 부호 규약
- 2.5 듀얼코어 태스크 배치

3. 저장소 디렉터리 구조와 폴더별 역할

- 3.1 최상위 레이아웃
- 3.2 폴더별 책임 요약
- 3.3 수명주기 분류 체계

4. 주력 비행 펌웨어: `dual_imu_cascade_pwm`

- 4.1 파일 구성과 빌드 단위
- 4.2 튜닝 파라미터와 시스템 상수
- 4.3 자료구조와 전역 상태
- 4.4 유틸리티 함수군
- 4.5 코어 간 동기화 함수군
- 4.6 자기계 처리 함수군
- 4.7 모터 출력과 믹서
- 4.8 캘리브레이션
- 4.9 `pid_task` — 1kHz 제어 루프
- 4.10 `udp_task` — 통신 태스크
- 4.11 명령 파서 함수군
- 4.12 텔레메트리 송신 함수군
- 4.13 `setup`과 `loop`

5. 주력 펌웨어 동반 헤더 4종

- 5.1 `failsafe_land.h` — 자동착륙과 호버 추정
- 5.2 `yaw_command.h` — yaw 각속도 명령과 heading 잠금
- 5.3 `mag_yaw_fusion.h` — 자기계 yaw 융합 수학
- 5.4 `msp_sensor.h` — MSPv2 센서 프레임 파서

6. 보류 후보 펌웨어: `dual_imu_flix_quat_pwm`

- 6.1 설계 차이 요약
- 6.2 수학 라이브러리 헤더
- 6.3 추정·제어 함수군
- 6.4 주력과의 비호환 지점

7. 진단 펌웨어: `firmware/diagnostics`

7.1 모터 계열 진단 3종

7.2 IMU 계열 진단 4종

8. 보관 펌웨어: `firmware/archive`

8.1 보관 정책

8.2 계열별 목록과 위험 표시

9. PC 지상국과 분석 도구: `scripts/`

9.1 도구 지형도와 동시 실행 금지 규칙

9.2 `telemetry_schema.py` — 공유 프로토콜 파서

9.3 `control_dualsense.py` — 주력 대화형 지상국

9.4 `failsafe_telemetry.py` — 자동착륙 상태 표시

9.5 `decode_imu_raw.py` — 바이너리 디코더

9.6 `magcal_fit.py` — 자기계 타원체 피팅

9.7 분석 도구 2종

9.8 수신·모니터 도구 3종

9.9 벤치 자동화 3종

9.10 보조 도구와 보관 도구

10. 검증 도구: `tools/`

10.1 2계층 검증 구조

10.2 네이티브 테스트 6종

10.3 Python 테스트 13종

10.4 `check_repo_layout.py`

11. 문서와 로그: `docs/`, `logs/`

11.1 문서 체계

11.2 로그 디렉터리

12. 빌드·CI·설정 파일

12.1 GitHub Actions 워크플로

12.2 의존성과 루트 설정 파일

13. 부록

13.1 텔레메트리 65필드 전체 명세

13.2 UDP 명령어 전체 목록

13.3 안전 규칙 요약

13.4 주요 함수 색인

13.5 코드 분석에서 발견한 구조적 관찰과 개선 제안

1. 문서 개요와 읽는 법

이 문서는 ZETIN Drone 저장소의 전체 코드를 폴더 → 파일 → 함수의 세 단계로 분해해, 각 단위가 무엇을 책임지는지(역할)와 어떤 원리로 그것을 달성하는지(작동 기제)를 기술한다. 신규 참여자가 이 문서만으로 시스템의 전체 구조와 개별 코드의 의도를 파악하고, 안전하게 수정 지점을 찾는 것이 목표다.

1.1 이 문서의 구성 원칙

원본 요청은 "폴더 역할 → 파일 역할 → 함수 역할 → 작동 기제" 순서였으나, 이 시스템은 **하나의 제어 루프를 중심으로 모든 코드가 방사형으로 연결된 구조**다. 파일을 알파벳순으로 나열하면 각 함수가 왜 존재하는지 설명할 수 없다. 따라서 다음과 같이 순서를 조정했다.

- 먼저 시스템 전체를 세운다**(2장) — 하드웨어, 데이터 흐름, 좌표계, 태스크 배치. 이 맥락 없이는 펌웨어의 부호 변환 한 줄도 해석할 수 없다.
- 폴더 구조를 개괄한다**(3장) — 어떤 폴더가 현행이고 어떤 폴더가 역사 보존용인지 먼저 구분한다.
- 중요도 순으로 코드를 깊이 판다**(4~10장) — 주력 비행 펌웨어를 가장 깊게, 보관 코드는 목록 수준으로. 실제로 비행하는 코드에 지면을 집중한다.
- 부록에 참조표를 모은다**(13장) — 텔레메트리 필드, UDP 명령, 함수 색인은 본문에서 찾기 어려우므로 별도 정리했다.

또한 요청에 따라 **원문에 없던 내용을 보강**했다. 구체적으로 13.5절의 구조적 관찰과 개선 제안, 4.9절의 제어 루프 단계별 시퀀스 재구성, 2.3절의 좌표 변환 유도, 각 장의 상호 참조가 그것이다.

1.2 표기 규약

표기	의미
주력	현재 실제 비행에 사용하는 지원 경로. 변경 시 가장 높은 주의가 필요하다.
보류	빌드는 되지만 실기 검증이 끝나지 않은 후보. 비행에 사용하지 않는다.
진단	벤치에서 하드웨어를 확인하는 용도. 비행 제어를 하지 않는다.
보관	역사 보존용. 빌드 대상이 아니며 수정하지 않는다.
역할	그 단위가 시스템에서 담당하는 책임.
기제	그 책임을 달성하는 구체적 알고리즘·자료구조·제약.
파일:행	저장소 기준 상대 경로와 행 번호.

검증 상태와 구현 상태의 구분

이 저장소의 문서들은 "구현됨"과 "실기에서 검증됨"을 엄격히 구분하는 규율을 지키고 있으며, 이 분석서도 같은 규율을 따른다. 예를 들어 자동착륙 상태기제는 **완전히 구현되어 있으나**, 착지 감지는 **미해결**이고 현재 동작은 시간 기반 하강이다. 코드에 존재한다는 사실이 그 기능이 안전하게 동작한다는 뜻은 아니다.

2. 시스템 아키텍처 조감

2.1 무엇을 만드는 프로젝트인가

센서 읽기, 자세 추정, PID 제어, 모터 출력, 안전장치, 그리고 PC 지상국까지 상용 비행 컨트롤러(Betaflight, ArduPilot 등)를 쓰지 않고 **전부 직접 구현하는 쿼드콥터 프로젝트**다. 최종 목표는 실내에서 안정적으로 조종되는 기체다.

현재 도달한 지점은 다음과 같다. 자세 제어(캐스케이드 PID)와 안전 경로는 공중에서 동작함이 2026-08-01 비행으로 확인되었고, yaw 자동 잠금과 자기계 보정도 구현·검증되었다. 그러나 **고도와 위치를 측정하는 센서가 페루프에 들어가지 않다**. 따라서 조종 스틱을 중립에 두어도 기체는 계속 흘러가며, 조종자가 수동으로 위치를 유지해야 한다. 거리계·광류 센서는 장착되어 있으나 텔레메트리 기록 전용이고 어떤 제어 판정에도 사용되지 않는다.

2.2 하드웨어 구성

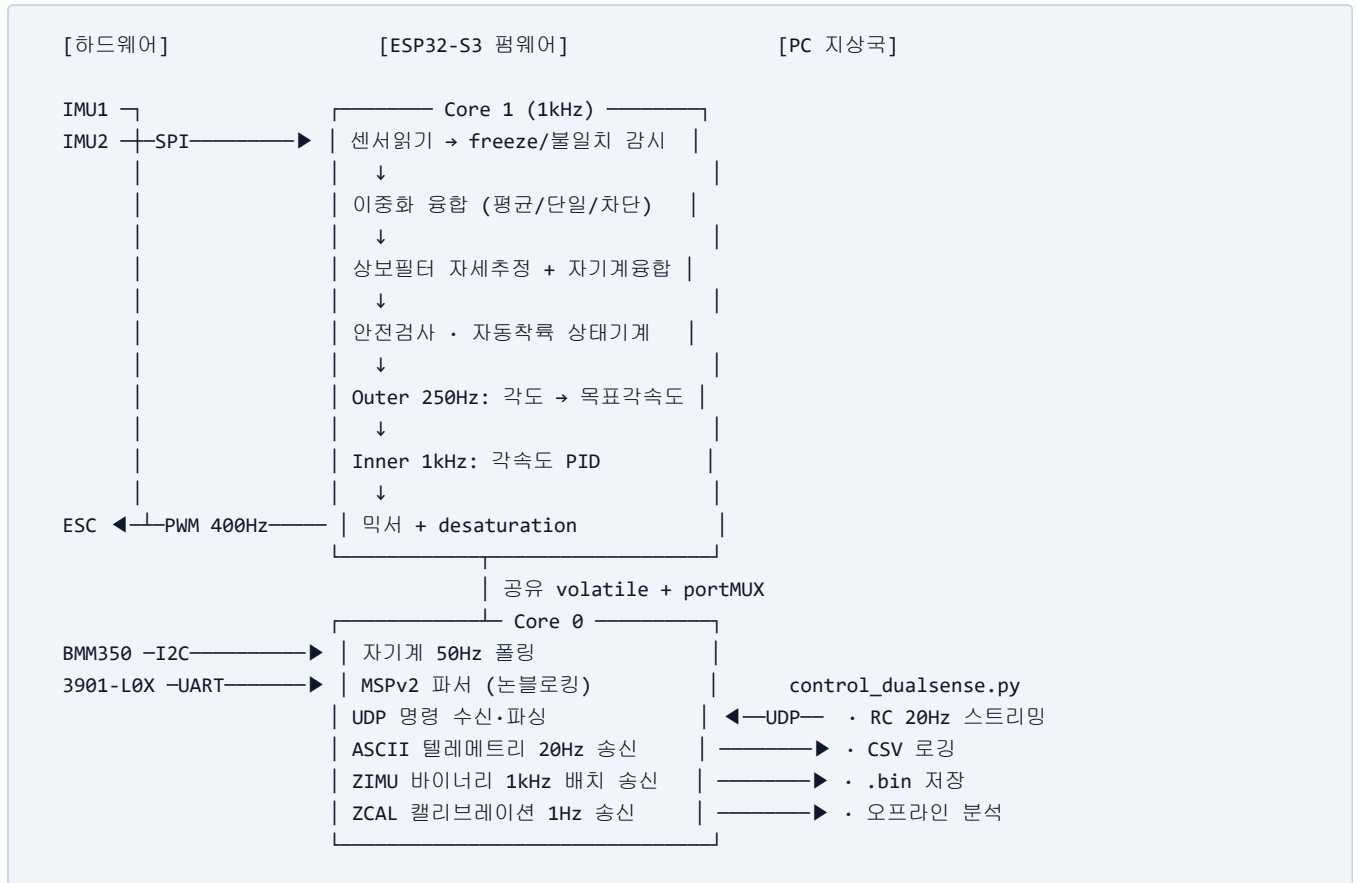
구성 요소	부품	연결과 역할
비행 컨트롤러 MCU	ESP32-S3	듀얼코어 Xtensa LX7. Core 1이 1kHz 제어 루프, Core 0이 WiFi/UDP/센서 폴링을 담당한다. WiFi SoftAP를 내장해 별도 통신 모듈이 필요 없다.
관성 센서 ×2	ICM-42670-P	SPI. SCK=12, MISO=13, MOSI=11, CS1=10, CS2=9. 자이로 $\pm 2000\text{dps}$, 가속도 $\pm 16\text{g}$, ODR 1600Hz. 이중화 구성으로 한쪽이 고장나도 비행을 유지한다.
자기계	BMM350	I2C 주소 0x14, SDA=35, SCL=36, LDO 셋다운 핀 38. 50Hz. yaw 적분 드리프트 보정에 선택적으로 사용(펌웨어 기본값 OFF).
광류 + 거리계	Matek 3901-L0X	PMW3901 광류 + VL53L0X ToF. 모듈 TX → GPIO16 수신 전용, 115200bps, INAV MSPv2 프레임. 전원은 반드시 5V . 현재 기록 전용.
ESC ×4	BLHeli 계열	PWM 400Hz, 14bit LEDC. M1=GPIO4(FL), M2=GPIO5(RR), M3=GPIO6(FR), M4=GPIO7(RL). 펄스폭 1000~2000 μs .
지상국	PC + DualSense	USB 게임패드를 PC에 연결하고, PC는 드론의 SoftAP <code>Drone_Tuning</code> 에 접속해 UDP 4210으로 통신한다.

단일 IMU 진단만 다른 핀 배치를 쓴다

`icm42670_single_raw` 진단은 SCK/MISO/MOSI/CS = 18/19/23/5를 사용한다. 비행 펌웨어(12/13/11, CS 10/9)와 배선이 다르므로 혼동하면 센서를 읽지 못한다.

2.3 소프트웨어 구성 요소와 데이터 흐름

시스템은 크게 네 덩어리다. 각각이 어떤 데이터를 주고받는지 따라가면 전체 구조가 잡힌다.



데이터 흐름의 네 경로

- 제어 경로(1kHz, Core 1 내부 완결)** — IMU SPI 읽기부터 ESC PWM 출력까지 한 태스크 안에서 끝난다. 외부 통신에 의존하지 않으므로 WiFi가 끊어져도 제어 루프 자체는 계속 돈다.
- 명령 상향 경로(20Hz, PC → 드론)** — 게임패드 입력이 `rcr <seq> <roll> <pitch> <yaw_rate>` 문자열로 UDP 전송된다. 드론은 이 패킷의 도착 시각으로 RC 위치독을 갱신한다.
- 텔레메트리 하향 경로(20Hz, 드론 → PC)** — 65개 필드의 CSV 형식 ASCII 문자열. PC가 수신 시각을 앞에 붙여 66열 CSV로 기록한다.
- 고속 원시 데이터 경로(1kHz 샘플 → 20Hz 배치, 드론 → PC)** — IMU 레지스터 원값을 512칸 링버퍼에 쌓고 최대 50샘플씩 `ZIMU` 바이너리 데이터그램으로 전송한다. 20Hz ASCII로는 잡히지 않는 현상을 오프라인에서 분석하기 위한 경로다.

텔레메트리 목적지가 재지정되는 구조

펌웨어는 들어온 모든 UDP 패킷의 발신자 주소로 텔레메트리 목적지를 다시 설정한다

(`dual_imu_cascade_pwm.ino:2021-2023`). 따라서 보조 도구가 명령을 하나라도 보내는 순간 텔레메트리를 가로챈다.

이것이 "지상국 도구를 동시에 둘 이상 실행하면 안 된다"는 규칙의 기술적 근거다.

2.4 좌표계와 부호 규약

이 프로젝트에서 가장 오해하기 쉬운 부분이다. 세 개의 서로 다른 좌표계가 존재하며, 두 비행 펌웨어가 서로 다른 규약을 쓴다.

(1) IMU2 → IMU1 센서축 정렬

두 IMU는 보드에 서로 다른 방향으로 실장되어 있다. IMU2는 IMU1 기준으로 X축과 Z축의 부호가 반대이고 Y축만 같다. 두 센서를 평균하기 전에 IMU2에 다음 부호를 곱한다.

```
IMU2_SIGN = { X: -1.0, Y: +1.0, Z: -1.0 }
```

(2) 융합 센서축 → 기체축 (주력 펌웨어, dual_imu_cascade_pwm)

```
body roll rate X = sensor Y      body accel X = sensor Y
body pitch rate Y = -sensor X     body accel Y = -sensor X
body yaw rate Z = -sensor Z       body accel Z = sensor Z
```

코드 위치: `dual_imu_cascade_pwm.ino:1371-1376`. 이 변환은 센서축을 Z 기준으로 회전시킨 것에 해당하며, yaw는 CW를 양으로 잡는다.

(3) 융합 센서축 → 기체축 (보류 후보, dual_imu_flix_quat_pwm)

```
body = ( -sensor_y, +sensor_x, +sensor_z )    // 오른손 FLU
roll+ = 오른쪽이 내려감
pitch+ = 기수가 내려감
yaw+ = 위에서 볼 때 CCW ← 주력과 부호가 반대
```

코드 위치: `dual_imu_flix_quat_pwm.ino:134-136` 의 `sensorToBody()`.

두 펌웨어의 yaw 부호가 반대다

주력 펌웨어는 yaw CW를 양으로, 보류 후보는 오른손 좌표계에 맞춰 CCW를 양으로 잡는다. 지상국의 yaw 명령을 그대로 두 펌웨어에 보내면 **회전 방향이 반대로 나온다**. 게인 단위도 다르다(주력은 $\mu\text{s}/(\text{deg/s})$, 보류 후보는 정규화 토크). 두 펌웨어는 서로 게인이나 명령을 이식할 수 없다.

(4) 모터 믹서 부호

M1/M2/M3/M4는 FL(전-좌)/RR(후-우)/FR(전-우)/RL(후-좌) 순서다. T를 collective throttle, R/P/Y를 roll/pitch/yaw 보정량이라 하면 주력 펌웨어의 차동 부호는 다음과 같다.

```
M1 (FL) = T - P + R - Y      프로펠러 회전: CW
M2 (RR) = T + P - R - Y      프로펠러 회전: CW
M3 (FR) = T - P - R + Y      프로펠러 회전: CCW
M4 (RL) = T + P + R + Y      프로펠러 회전: CCW
```

2.5 듀얼코어 태스크 배치

태스크	코어	우선 순위	주기	책임
<code>pid_task</code>	Core 1	2	1ms (1kHz)	IMU 읽기, 융합, 자세 추정, 안전 판정, 자동착륙, 캐스케이드 PID, 믹서, PWM 출력. 모터를 쓰는 유일한 태스크.

태스크	코어	우선 순위	주기	책임
udp_task	Core 0	1	1 tick	UDP 패킷 파싱, 명령 처리, 자기계 I2C 폴링, MSP UART 폴링, 텔레메트리 3종 송신.
loop()	Core 1 (Arduino)	1	1초	평시에는 <code>delay(1000)</code> 만 한다. 지연 디버그 빌드에서만 링버퍼를 비운다.

이 배치가 만드는 세 가지 설계 제약이 코드 전반을 지배한다.

1. **블로킹 I/O는 Core 0에만 둔다.** BMM350의 I2C 읽기는 블로킹이므로 `udp_task` 에서만 호출한다. 과거에 이 호출이 RC 수신 큐를 넘치게 한 전례가 있어, MSP UART 폴링도 반복당 64바이트 상한을 둔 논블로킹 구조로 되어 있다.
2. **모터 PWM의 writer는 반드시 하나여야 한다.** 두 코어가 동시에 쓰면 경합이 발생하므로 `pid_task` 만 `writeMotor()` 를 호출한다. Core 0은 `safety_arm_requested` 같은 플래그로 요청만 게시하고, 실제 상태 전이는 Core 1이 수행한다.
3. **태스크 위치독이 최후의 안전장치다.** 정지 수단(`stopMotors` , RC 타임아웃 검사)이 전부 `pid_task` 안에 있으므로, SPI 행업 등으로 이 태스크가 멈추면 모터가 마지막 PWM으로 고정된다. 500ms 태스크 위치독이 panic 재부팅을 강제해 그 상태에서 벗어난다.

3. 저장소 디렉터리 구조와 폴더별 역할

3.1 최상위 레이아웃

Drone_github_DoHyun/	
└─ firmware/	ESP32-S3 등 MCU 펌웨어 (Arduino 스케치 26개)
│ └─ flight/	비행 제어 후보 2개 [주력 1 + 보류 1]
│ └─ diagnostics/	현행 벤치 진단 7개 [진단]
│ └─ archive/	미지원 역사 펌웨어 17개 [보관]
└─ scripts/	PC 지상국·분석 도구 (Python 16개)
│ └─ archive/	폐기된 GPS/TCP 도구 2개 [보관]
└─ tools/	저장소·알고리즘 검증 (Python 14 + C++ 6)
│ └─ native_tests/	호스트 단위검사·SIL
│ └─ shims/	Arduino API 대역 헤더 7종
└─ docs/	설계·절차·프로토콜 문서
│ └─ design/ plans/	설계와 실행 계획
│ └─ history/	과거 설계 기록 (보존)
│ └─ reports/	정합성 점검 보고
│ └─ superpowers/	기능별 설계·계획 (specs/ plans/)
│ └─ archive/	보관 스니펫
└─ logs/	비행 CSV·IMU raw 로그 저장 위치
└─ .github/workflows/	CI 정의 2개

3.2 폴더별 책임 요약

폴더	분류	역할과 유지 규칙
firmware/flight/	주력	실제 기체에 올리는 비행 제어 코드. CI 컴파일 대상. 두 개의 후보가 있으며 하나만 주력이다.
firmware/diagnostics/	진단	프로펠러를 제거한 벤치에서 하드웨어를 한 요소씩 확인하는 스케치. PID나 믹서를 포함하지 않는다. CI 컴파일 대상.
firmware/archive/	보관	대체된 펌웨어를 원형 그대로 보존한다. 빌드하지 않고 수정하지 않는다. 안전하다고 가정해서도 안 된다.
scripts/	주력	PC에서 실행하는 조종·수신·기록·분석 도구. 펌웨어와 UDP 프로토콜로 결합되어 있어 필드 순서 변경 시 함께 수정해야 한다.
tools/	주력	기체 없이 PC에서 돌리는 검증 계층. 펌웨어 헤더를 호스트에서 직접 컴파일해 제어 수학을 단위검사하고, 페루프 SIL까지 수행한다.
docs/	혼합	현행 문서, 절차서, 프로토콜 규격, 그리고 역사 기록이 함께 있다. history/ · archive/ 의 내용을 현재 상태의 근거로 쓰지 않는다.
logs/	산출물	비행 CSV(flight_log_*.csv)와 IMU raw(imuraw_*.bin/.csv)가 생성되는 곳. README만 추적하고 로그 자체는 대부분 무시한다.

3.3 수명주기 분류 체계

이 저장소의 가장 두드러진 특징은 **모든 코드에 수명주기 라벨이 붙어 있고 그것이 문서로 강제된다는 점**이다.

`docs/firmware_catalog.md` 가 26개 스케치 전부를 주력 1 / 보류 1 / 진단 7 / 보관 17로 분류하고, CI는 `flight/` 와 `diagnostics/` 만 컴파일한다. 나아가 `tools/check_repo_layout.py` 가 문서 링크의 유효성, 스케치 디렉터리명과 파일명의 일치, 마이그레이션 맵 44행의 정합성, 폐기된 경로명의 잔존 여부를 자동 검사한다.

이 구조가 주는 실질적 이득

신규 참여자가 "이 파일을 고쳐도 되는가"를 코드를 읽지 않고 **경로만 보고** 판단할 수 있다. `archive/` 아래면 손대지 않고, `flight/` 아래면 최고 수준의 검토가 필요하다. 26개 스케치가 뒤섞여 있었다면 불가능했을 판단이다.

4. 주력 비행 펌웨어: dual_imu_cascade_pwm

firmware/flight/dual_imu_cascade_pwm/ **주력**

이 저장소에서 유일하게 실비행에 사용하는 코드다. 본체 `dual_imu_cascade_pwm.ino` 가 2,360행으로 전체 소스의 가장 큰 단일 파일이며, 나머지 코드 대부분은 이 파일을 지원하거나 검증하기 위해 존재한다. 따라서 이 장이 문서 전체에서 가장 상세하다.

4.1 파일 구성과 빌드 단위

Arduino 스케치는 디렉터리 단위로 빌드되므로, 같은 폴더의 `.h` 파일 4개가 함께 컴파일된다. 단일 `.ino` 가 아니라 5개 파일이 하나의 빌드 단위다.

파일	행수	역할
<code>dual_imu_cascade_pwm.ino</code>	2,360	본체. 상수, 전역 상태, 두 FreeRTOS 태스크, 명령 파서, 텔레메트리 송신, 초기화를 모두 포함한다.
<code>failsafe_land.h</code>	253	자동착륙 상태기계, 호버 스로틀 추정기, 능동 프로브 착지 검출기, resume 거부 사유 열거.
<code>msp_sensor.h</code>	192	Matek 3901-L0X가 보내는 INAV MSPv2 프레임의 바이트 단위 상태기계 파서.
<code>mag_yaw_fusion.h</code>	41	각도 래핑, 틸트 보상 자기 heading 계산, yaw 보정량 계산.
<code>yaw_command.h</code>	39	yaw 각속도 명령 모드와 heading 자동 잠금 판정.

왜 헤더로 분리했는가 — 단순한 정리가 아니다

두 가지 실무적 이유가 있다. 첫째, Arduino 전처리기는 `.ino` 의 첫 함수 정의 앞에 자동 생성한 함수 원형을 삽입한다. 열거형이나 구조체를 `.ino` 안에 두면 그 원형이 타입 선언보다 앞서 삽입되어 컴파일이 깨진다. 헤더는 이 전처리 대상이 아니므로 안전하다 (`failsafe_land.h:102-105` 에 이 사정이 주석으로 남아 있다). 둘째, 헤더로 분리해야 `tools/native_tests/` 가 이 순수 함수들을 PC에서 `g++` 로 직접 컴파일해 단위검사를 할 수 있다.

4.2 튜닝 파라미터와 시스템 상수

제어 게인 (런타임 변경 가능, `volatile`)

변수	기본값	의미와 단위
<code>Kp_Angle_Roll/Pitch</code>	6.0	바깥 루프 각도 P. 각도 오차(deg) → 목표 각속도(dps)로 변환하는 비례 게인.
<code>Kp_Angle_Yaw</code>	3.0	heading 잠금 시 yaw 각도 P.
<code>Kp_Rate_Roll/Pitch</code>	0.50	안쪽 루프 각속도 P. 각속도 오차(dps) → 모터 출력(μ s).
<code>Ki_Rate_Roll/Pitch</code>	0.05	각속도 I. 적분 기여를 μ s 단위로 누적하며 $\pm 50\mu$ s로 제한한다.

변수	기본값	의미와 단위
Kd_Rate_Roll/Pitch	0.015	각속도 D.
Kp/Ki/Kd_Rate_Yaw	1.50 / 0.05 / 0.0	yaw 각속도 PID.

Ki 값이 0.005에서 0.05로 바뀐 근거가 코드에 남아 있다

주석(:144-148)에 따르면, 0.005는 일정한 CG 치우침 외란에서 10초가 지나도 $\pm 50\mu\text{s}$ 클램프의 약 1.4%만 사용해 **적분기가 사실상 비활성**이었다. 호스트 SIL로 이를 측정한 뒤 0.05로 올렸고, 이는 적분기를 활성화하되 클램프의 $5\mu\text{s}$ 미만만 쓰는 보수적 값이다. SIL이 제시한 완전 트림 값(0.5)은 실기의 지연과 노이즈에서 진동 위험이 있어 **의도적으로 피했다**. 이런 식으로 이 코드베이스는 상수 하나하나에 측정 근거와 기각된 대안을 함께 기록한다.

주요 시스템 상수

상수	값	역할
WIFI_SSID / PASS	Drone_Tuning / 12345678	SoftAP 자격. 드론 IP는 192.168.4.1.
WIFI_CHANNEL	11	ch6은 인근 AP가 많아 혼잡해 스캔상 가장 빈 채널로 이동했다.
UDP_PORT	4210	명령 수신과 텔레메트리 송신에 공용.
ESC_FREQ / RES / PERIOD	400Hz / 14bit / 2500 μs	LEDC PWM. duty = us $\times$ 16383 / 2500.
GYRO_SCALE	1/16.4	$\pm 2000\text{dps}$ FSR의 raw LSB $\rightarrow$ deg/s.
ACCEL_SCALE	1/2048	$\pm 16\text{g}$ FSR의 raw LSB $\rightarrow$ g.
SAFETY_ANGLE	60.0 $^{\circ}$	roll 또는 pitch가 이를 넘으면 즉시 잠금.
MAX_TARGET_ANGLE_RP	30.0 $^{\circ}$	UDP 오입력에 대한 자세 명령 상한(트림 포함 후 클램프).
MAX_TARGET_RATE_RP / YAW	300 / 180 dps	바깥 루프 출력 제한.
TRIM_MAX_DEG	10.0 $^{\circ}$	트림 절대값 상한.
RC_TIMEOUT_MS	500	이보다 오래 RC가 없으면 failsafe 진입.
CTRL_MARGIN	150	base_throttle 기준 상하 제어 여유(μs).
PID_WDT_TIMEOUT_MS	500	pid_task 태스크 위치독. 초과 시 panic 재부팅.
OUTER_DIV	4	1kHz $\div$ 4 = 250Hz 바깥 루프.
GYRO_SW_LPF_HZ	80.0	하드웨어 LPF(121Hz)에 겹치는 소프트웨어 LPF.

상수	값	역할
I_TERM_MAX_US	50.0	축별 적분 기여 상한(μ s).
FROZEN_DELTA_RAW	1 LSB	6축 raw 변화량 합이 이 이하면 정지 의심.
IMU_FROZEN_MS	300	그 상태가 이만큼 지속되면 freeze 확정.
GYRO_DISAGREE_DPS	15.0	두 IMU 각속도 차이 임계.
IMU_DISAGREE_MS	150	불일치 지속 시간.

컴파일 타임 불변식 — `static_assert` 방어선

이 펌웨어는 안전 관련 상수들 사이의 관계를 **컴파일 타임에 강제한다**. 이 어서션들은 단순한 형식 검사가 아니라 실제 사고 분석에서 도출된 조건이다.

어서션	보장하는 것
<pre>sizeof(ImuRawSample) == 27 sizeof(ImuRawHeader) == 20 sizeof(ImuRawDatagram) == 1370 sizeof(ImuCalDatagram) == 60</pre>	바이너리 wire 포맷이 PC 디코더의 <code>struct.Struct</code> 정의와 바이트 단위로 일치함. 구조체에 필드를 추가하면 여기서 즉시 깨진다.
<pre>FS_DESCENT_DELTA_US < CTRL_MARGIN</pre>	하강값이 제어 여유보다 작아 하강 중에도 자세 제어 권한이 남음.
<pre>FS_PROBE_DIP_MS * FS_LAND_LPF_ALPHA ≥ 3.0</pre>	프로브 딥이 최소 3τ (LPF 95% 응답) 동안 지속됨.
<pre>FS_PROBE_DIP_FRAC * 0.95 > 1.5 * FS_PROBE_RESPONSE_G</pre>	관측 가능한 프로브 응답이 판정 임계의 1.5배를 넘음.
<pre>FS_PROBE_MIN_DELIVERY_FRAC * FS_PROBE_DIP_FRAC * 0.95 > 1.2 * FS_PROBE_RESPONSE_G</pre>	최소 전달률(80%)에서도 응답 여유가 보존됨.
<pre>HOVER_MIN_THROTTLE_US ≥ 1000 + FS_DESCENT_DELTA_US + FS_PROBE_DIP_MIN_US</pre>	호버 추정 하한이 프로브 딥을 아이들 위로 유지함.
<pre>FS_MIN_DESCEND_MS + FS_PROBE_PERIOD_MS * FS_PROBE_CONFIRM_N + FS_PROBE_DIP_MS < FS_MAX_MS</pre>	백스톱 컷 전에 착지 확정이 가능한 시간 배치(필요조건). 이 식은 <code>confirm_n-1</code> 로 잘못 쓰면 통과하지만 SIL에서 3건 타임아웃이 났고, 변조 검사로 그 오차를 발견해 고친 것이다.

4.3 자료구조와 전역 상태

바이너리 wire 구조체 4종

구조체	크기	내용과 목적
ImuRawSample	27 B	dt_us (u16), IMU1/IMU2 각각의 gyro-accel 3축 raw(i16×12), failsafe_phase (u8). v2에서 phase가 추가됐다.
ImuRawHeader	20 B	magic "ZIMU", version, n_samples, reserved, batch_seq, base_t_us, dropped.
ImuRawDatagram	1,370 B	헤더 + 최대 50샘플. 실제 전송 길이는 샘플 수에 맞춰 계산한다.
ImuCalDatagram	60 B	magic "ZCAL", 두 IMU의 자이로 bias·가속도 스케일, 스케일 상수, IMU2 부호. 1Hz로 보내며 이것 없이는 raw를 물리량으로 복원할 수 없다.

failsafe_phase 를 v2에서 추가한 이유

주석(:29-32)이 문제를 정확히 기록하고 있다. 1kHz .bin 로그는 펌웨어의 micros() 기준이고 20Hz CSV는 PC 벽시계 기준이라 두 로그에 공통 키가 없었다. 그 결과 2026-08-01 분석에서 1kHz 데이터를 자동착륙 위상으로 자를 수 없어 착지 판별식 후보 평가가 막혔다. 샘플마다 phase를 1바이트 넣는 것으로 이 문제를 해결했다.

락 없는 링버퍼 ImuRawRing

Core 1(생산자)이 1kHz로 밀어넣고 Core 0(소비자)이 20Hz로 배치를 꺼내 간다. 512칸(2의 거듭제곱)이며 인덱스를 마스킹해 모듈로 연산을 피한다. 동기화는 뮤텍스 없이 GCC 원자 내장함수로 처리한다.

```
head 갱신: __atomic_store_n(&ring.head, head + 1, __ATOMIC_RELEASE)
head 읽기: __atomic_load_n(&ring.head, __ATOMIC_ACQUIRE)
```

release/acquire 쌍이 샘플 데이터 쓰기가 인덱스 갱신보다 먼저 보이도록 보장한다. 큐가 가득 차면 dropped 를 올리고 새 샘플을 버린다(오래된 샘플을 덮어쓰지 않는다). 그 카운터를 데이터그램 헤더에 실어 PC가 유실을 인지한다.

전역 상태 변수 분류

그룹	writer	주요 변수
추정 자세·센서	Core 1	angleX/Y/Z, gyroX/Y/Z, accX/Y/Z — 융합 후 기체축 값.
목표값	Core 0 (명령)	targetAngleX/Y/Z, targetYawRate, trim_roll/pitch.
출력	Core 1	motorOut[4], tgtRate[3], pidLoopHz.
고장 래치	Core 1	fault_rc, fault_imu1/2, fault_disagree, fault_attitude, active_imus.
현재 센서 상태	Core 1	imu1/2_frozen_now, imu_disagree_now — 래치와 달리 매 tick 갱신되며 재시동 판단에 쓴다.

그룹	writer	주요 변수
안전 요청	Core 0 → Core 1	safety_arm/disarm_requested, hover_reset_requested, failsafe_resume_requested — safetyRequestMux 로 보호.
자동착륙	Core 1	fs_phase, fs_probe_state, fs_probe_no_response, hover_est, hover_valid.
자기계	Core 0 → Core 1	magFieldX/Y/Z, magHeading, mag_enabled — magSnapshotMux 로 보호.
3901-L0X	Core 0	mzp_range_mm, mzp_range_quality, mzp_flow_x/y, mzp_flow_quality.

고장 래치와 현재 상태를 분리한 설계

fault_imu1 은 비행 중 자동 복구하지 않는다. 한 번 freeze로 판정된 센서는 값이 다시 움직여도 자동 재투입하지 않고, 다음 수동 start 때만 재평가한다. 반면 imu1_frozen_now 는 매 tick 현재 상태를 반영해 "지금 이 센서로 시동을 걸어도 되는가"를 판단하는 데 쓴다. 간헐 고장 센서가 비행 중 제어에 다시 들어왔다 나갔다 하는 것을 막는 구조다.

4.4 유틸리티 함수군

```
class LowPassFilter { float alpha, last; update(in); reset(value); }
```

역할 — 1차 IIR 저역통과 필터. D항, 자이로, 착지 검출에 공통 사용.

기제 — 생성자에서 차단주파수와 dt로 $rc = 1/(2\pi \cdot f_c)$, $\alpha = dt/(rc+dt)$ 를 계산해 둔다. update() 는 $last += \alpha \cdot (in - last)$ 한 줄이다. 나눗셈이나 삼각함수가 없어 1kHz 루프에서 비용이 무시할 만하다.

```
static inline uint32_t imuRawRingCount(const ImuRawRing &ring)
```

역할 — 링버퍼에 쌓인 샘플 수를 반환.

기제 — head - tail 을 부호 없는 뺄셈으로 계산한다. 두 인덱스가 절대 리셋되지 않고 단조 증가만 하므로, uint32 wrap이 일어나도 모듈러 뺄셈이 정확한 개수를 준다.

```
static inline bool imuRawRingPush(ImuRawRing &, const ImuRawSample &, uint32_t sampleUs)
```

역할 — 생산자(Core 1)가 샘플 하나를 큐에 넣는다.

기제 — 가득 찼으면 dropped 를 원자적으로 증가시키고 false 를 반환한다(새 샘플 폐기). 큐가 비어 있다가 첫 샘플이 들어가는 순간 first_t_us 에 절대 시각을 기록한다 — 소비자는 이 값 하나만으로 이후 샘플들의 시각을 dt_us 누적으로 복원한다.

```
static inline bool imuRawRingPop(ImuRawRing &, ImuRawSample &)
```

역할 — 소비자(Core 0)가 샘플 하나를 꺼낸다.

기제 — tail == head 면 비어 있으므로 false . 아니면 복사 후 tail을 release 저장으로 전진시킨다.

```
static inline ImuRawSample makeImuRawSample(uint16_t dtUs, const inv_imu_sensor_event_t &e1, const inv_imu_sensor_event_t &e2, uint8_t failsafePhase)
```

역할 — 두 IMU의 raw 이벤트를 27바이트 wire 샘플로 포장.

기제 — 부호 보정·스케일·bias 보정을 **전혀 하지 않고** 레지스터 값을 그대로 담는다. 보정 상수는 ZCAL 로 따로 보내므로, PC에서 나중에 다른 보정 가설을 적용해 재해석할 수 있다.

```
static inline size_t buildImuRawDatagram(ImuRawDatagram &, uint32_t batchSeq, uint32_t baseTUs, uint32_t dropped, const ImuRawSample *, uint8_t nSamples)
```

역할 — 헤더를 채우고 샘플을 복사해 전송 길이를 반환.

기제 — nSamples 가 0이거나 50을 넘으면 0을 반환해 전송을 막는다. 반환값은 `sizeof(header) + n × sizeof(sample)` 이므로 고정 1370바이트가 아니라 실제 필요한 만큼만 보낸다.

```
static inline ImuTelemetrySample makeImuTelemetrySample(const float g1[3], const float a1[3], const float g2[3], const float a2[3])
```

역할 — IMU별 값을 센서측에서 기체측으로 변환해 텔레메트리 구조체로.

기제 — 2.4절의 변환을 그대로 적용한다: $(g[1], -g[0], -g[2])$, 가속도는 여기에 ACCEL_SCALE 을 곱한다. **변환 계산을 임계구역 밖에서 끝내고**, 락 안에서는 float 12개만 복사하도록 설계되어 Core 1의 락 보유 시간을 최소화한다.

```
static bool checkFreeze(FreezeMon &m, const inv_imu_sensor_event_t &e, uint32_t nowMs)
```

역할 — IMU 레지스터 값이 멈췄는지(통신 정지·동일 프레임 반복) 감시.

기제 — 6축 각각의 **절대 변화량을 더한다** $(\sum |e.gyro[k] - last[k]| + \sum |e.accel[k] - last[k]|)$. 주석이 명시하듯, $|x|+|y|+|z|$ 의 변화만 보면 축 간 변화가 서로 상쇄되어 정지를 놓칠 수 있다. 합이 FROZEN_DELTA_RAW (1 LSB) 이하인 상태가 IMU_FROZEN_MS (300ms) 지속되면 freeze를 확정한다. 첫 호출은 기준값만 저장하고 false 를 반환한다. **읽기 오류로 event가 갱신되지 않는 경우도 자동으로 여기에 걸린다** — 읽기 실패를 별도로 처리할 필요가 없는 우아한 설계다.

```
static inline bool rcTimedOut(uint32_t nowMs, uint32_t lastMs)
```

역할 — RC 위치독. 마지막 RC 수신 이후 경과가 500ms를 넘었는지 판정.

기제 — $(int32_t)(nowMs - lastMs) > (int32_t)RC_TIMEOUT_MS$. **부호 있는 차이로 비교하는 것이 핵심이다.** nowMs 는 루프 맨 위에서 읽은 값이고 lastRcMs 는 Core 0이 그 뒤에 갱신할 수 있어, lastRcMs 가 nowMs 보다 **미래**일 수 있다. 부호 없는 뺄셈이면 언더플로로 거대한 양수가 되어 즉시 타임아웃이 터진다. 부호 있는 차이는 이 경우 음수가 되어 정상 처리되며, millis() wrap도 모듈러 뺄셈이라 그대로 안전하다.

```
static inline float compute_alpha(float ax, float ay, float az)
```

역할 — 가속도 신뢰도에 따라 상보필터 계수를 적응적으로 선택.

기제 — $dev = ||a| - 1.0|$ 를 구해 세 구간으로 나눈다. $dev < 0.10$ 이면 0.999(정지, $\tau \approx 1s$), < 0.30 이면 0.9995($\tau \approx 2s$), 그 이상이면 0.9998($\tau \approx 5s$). 가속도가 중력에서 멀수록 자이로를 더 신뢰한다. **1kHz 전환 시 값을 바꾼 이력이 중요하다** — 이전 값 0.99는 250Hz legacy 루프에서 온 것으로 τ 가 0.1초에 불과했다. 1kHz에서 그대로 쓰면 비행 중 가속도 (추력 방향 + 진동)에 자세 추정이 끌려가 호버가 불안정해진다.

```
static inline void enterLockedState(bool &wasLocked)
```

역할 — 잠금 진입 시 1회만 수행할 정리를 보장.

기제 — `wasLocked` 가 거짓일 때만 `yaw_hold_override` 를 해제하고 `wasLocked` 를 세운다. 세 개의 서로 다른 잠금 경로(IMU 전멸, 일반 `safety_lock`, PID 후단)가 모두 이 함수를 통해서만 플래그를 세운다. **매 tick 지우면 시동 해제 상태에서 yaw 1 명령을 켤 수 없게 된다** — 커자마자 다음 tick이 지워버리기 때문이다.

4.5 코어 간 동기화 함수군

ESP32-S3에서 32비트 읽기/쓰기는 원자적이지만, 여러 변수의 일관된 묶음을 주고받으려면 임계구역이 필요하다. 이 펌웨어는 세 개의 `portMUX_TYPE` 스핀락을 쓴다: `safetyRequestMux` (안전 요청), `magSnapshotMux` (자기계 스냅샷), `imuSampleMux` (IMU 텔레메트리).

```
static inline void requestSafetyDisarm() / requestSafetyArm()
```

역할 — Core 0이 시동/해제를 요청만 한다.

기제 — `disarm` 은 `arm` 요청과 `hover reset` 요청을 함께 지워 **disarm이 arm보다 무조건 우선**하게 만든다. `arm` 은 이미 `disarm`이 대기 중이면 아무것도 하지 않는다. 두 요청이 동시에 대기하는 경합에서 안전한 쪽이 이기는 구조다.

```
static inline void applyPendingSafetyRequest()
```

역할 — Core 1이 매 tick 맨 앞에서 대기 중인 요청을 실제 상태 전이로 변환.

기제 — 임계구역 안에서 `disarm`을 먼저 검사한다. `arm`의 경우, `hover_reset_requested` 가 함께 서 있으면 `hover_valid=false`, `hover_est=0`, `hoverTracker={}` 로 초기화한 **뒤에만** `safety_lock=false` 를 공개한다. 순서가 반대면 무장 직후 한 tick 동안 이전 비행의 호버 추정치가 살아 있는 창이 생긴다.

```
static inline ResumeRefusalReason resumeRefusalReason(uint32_t nowMs)
```

역할 — 자동착륙 중 `resume` 요청의 수락 여부와 거부 사유를 판정.

기제 — 6개 조건을 순서대로 검사하고 첫 실패 사유를 반환한다. ① 하강 중이 아니면 `PHASE`, ② 최초 진입으로부터 누적 9초 (`FS_RESUME_MAX_MS = 3×FS_MAX_MS`)가 지났으면 `CUMULATIVE`, ③ RC가 여전히 끊겼으면 `RC`, ④ 기울기 60° 초과면 `TILT`, ⑤ IMU 전멸·양쪽 freeze·불일치면 `IMU`, ⑥ 호버 추정 무효면 `HOVER`. 모두 통과해야 `RESUME_ALLOWED`. **누적 제한이 있는 이유**는 `resume`을 반복해 자동착륙을 무한정 연장하는 것을 막기 위해서다. 한 비행의 최초 진입 시각은 `resume`으로 지워지지 않는다.

```
static inline bool configChangeAllowed(const char *setting)
```

역할 — 자동착륙 하강 중 설정 변경을 일괄 차단.

기제 — `fs_phase != FS_DESCENDING` 이면 허용. 아니면 거부 로그를 찍고 `false`. PID 게인, 트림, 자기계 보정 계수 변경이 모두 이 게이트를 통과해야 한다. 하강 중 게인이 바뀌면 착륙 거동이 예측 불가능해지기 때문이다.

`publishMagSample / readMagSnapshot / publishMagTelemetry / readMagTelemetry`

역할 — 자기계 값을 Core 0에서 Core 1로 원자적으로 전달.

기제 — I2C 읽기(블로킹)를 임계구역 **밖**에서 끝내고, 락 안에서는 float 3개 + 타임스탬프만 복사한다. 그래서 pid_task가 블로킹 I2C 트랜잭션을 기다리는 일이 절대 없다. `publishMagTelemetry` 에는 미묘한 방어가 하나 더 있다 (:837-844): Core 1이 이미 정상 도메인 블록에 진입한 상태에서 Core 0이 보정을 시작할 수 있으므로, `calibration_raw || !mag_calibrating` 조건으로 **늦게 도착한 정상 도메인 writer가 raw 스냅샷을 덮어쓰지 못하게** 막는다.

`requestMagReferenceUpdate() / takeMagReferenceRequest()`

역할 — 자기 heading 기준 offset의 재설정을 요청·소비.

기제 — 게시-소비 플러그. 자기계를 켜거나 `angleZ` 를 0으로 리셋할 때 요청을 세우면, Core 1이 다음 자기계 융합 tick 에서 이를 소비하며 `magYawReferenceOffset = wrapDeg(angleZ - heading)` 을 다시 계산한다. **상대 heading 방식을 쓰는 이유**는 자기계를 켜고 기체가 자북으로 회전 명령을 받는 것을 막기 위해서다. 절대 방위가 아니라 "컬 때의 방향"을 기준으로 드리프트만 보정한다.

`publishImuSample / readImuSampleSnapshot / publishImuRawRegisters`

역할 — IMU별 값의 코어 간 전달과 raw 링버퍼 게시.

기제 — `publishImuRawRegisters` 는 직전 샘플과의 `dt_us` 를 계산해 담는데, 65535 μ s를 넘으면 0xFFFF 로 포화시킨다. 중요한 점은 **push가 성공했을 때만** `rawProducerLastUs` 를 갱신한다는 것이다. 큐가 가득 차 샘플이 버려진 경우 시간 기준을 전진시키지 않아야, 다음 성공 샘플의 `dt` 가 버려진 구간까지 포함해 누적 시각이 어긋나지 않는다.

4.6 자기계 처리 함수군

`static bool initMagnetometer()`

역할 — BMM350을 최초 1회 초기화.

기제 — LDO 켜다운 핀(GPIO38)을 HIGH로 올리고 100ms 대기 후 I2C를 시작한다. `begin()` 성공 시 Normal 모드 + 50Hz로 설정하고 `mag_ready` 를 세운다. `lastMagReadMs` 를 한 주기 과거로 설정해 첫 샘플이 즉시 찍히게 한다. **블로킹 delay가 있어** 무장 상태에서는 호출이 거부된다.

`static void startMagCalibration() / stopMagCalibration()`

역할 — hard/soft-iron 보정을 위한 raw 데이터 수집 구간의 시작·종료.

기제 — `start` 는 자기계 융합을 끄고 min/max 누산기를 `±FLT_MAX` 로 초기화한 뒤 `mag_calibrating` 을 세운다. 이 동안 텔레메트리 `Mag_X/Y/Z` 에는 **보정 전 raw μ T**가 나가고, `Mag_Cal_Active` (필드 65)가 1이 되어 PC가 도메인을 구분한다. `stop` 은 마지막 스냅샷에 throttle 간섭 보정을 적용해 되돌려 놓고, 수집 통계(샘플 수, 축별 span)를 출력한 뒤 `magcal_fit.py` 를 돌리라고 안내한다. **펌웨어는 피팅을 하지 않는다** — 수집만 하고 실제 타원체 피팅은 PC에서 NumPy 로 수행한다.

```
static inline void applyMagCalibration(const float raw[3], const float hard_iron[3], const float soft_iron[3][3], float out[3])
```

역할 — raw 자기장에 hard/soft-iron 보정과 기체축 부호를 적용.

기제 — 세 단계다. ① `centered = raw - hard_iron` (hard-iron: 기체에 고정된 자성체가 만드는 상수 오프셋 제거). ② `transformed = soft_iron × centered` (soft-iron: 주변 강자성체가 자기장을 타원형으로 왜곡하는 것을 3×3 행렬로 되돌림). ③ `out = body_sign ⊙ transformed` (센서축 → 기체축). 결과적으로 보정 후 자기장 벡터의 크기가 방향과 무관하게 일정해진다.

```
static void sampleMagnetometer(uint32_t now_ms)
```

역할 — 50Hz로 BMM350을 읽어 보정 후 게시. **Core 0 전용**.

기제 — `MAG_SAMPLE_PERIOD_MS` (20ms) 게이트로 호출 빈도를 제한한다. `isfinite` 검사로 이상값을 조심히 버린다. 보정 중이면 raw를 텔레메트리로 내보내고 min/max를 갱신한다. 그 뒤 `applyMagCalibration` 을 거친 값을 `publishMagSample` 로 게시한다. 주석이 강조하듯 이 블로킹 I2C 읽기의 호출자는 언제나 `udp_task`이며 `pid_task`가 아니다.

4.7 모터 출력과 믹서

```
void writeMotor(int pin, int us)
```

역할 — 펄스폭(μs)을 LEDC duty로 변환해 출력.

기제 — `constrain(us, 1000, 2000)` 으로 먼저 제한한 뒤 `duty = us × 16383 / 2500` . 정수 연산이며 오버플로를 피하려 `uint32_t` 로 캐스팅한다. 클램프가 변환보다 앞에 있는 순서가 중요하다 — 반대면 범위 밖 값이 duty 레지스터를 넘쳐 예측 불가능한 펄스가 나간다.

```
void stopMotors()
```

역할 — 네 모터를 모두 아이들(1000μs)로.

기제 — `writeMotor` 4회. 모든 안전 경로의 종착점이다.

```
static MotorMix mixAndDesaturate(float roll, float pitch, float yaw, int throttle, int minMotor, int maxMotor)
```

역할 — PID 출력 3축을 4모터 PWM으로 변환하면서, 포화 시 **자세 명령보다 collective를 먼저 희생**한다.

기제 — 이 펌웨어에서 가장 정교한 함수이며 4단계로 동작한다. ① **차동 계산** — 2.4절의 믹서 부호로 `diff[4]` 를 만든다. 이 값들은 collective를 뺀 순수 자세 차동이다. ② **공통 스케일 판정** — `span = max(diff) - min(diff)` 가 `available = maxMotor - minMotor` 보다 크면, 아무리 collective를 옮겨도 네 값을 범위 안에 넣을 수 없다. 이때만 `scale = available/span` 을 **세 축 모두에 동일하게** 곱한다. 축별로 다르게 자르면 토크 비율이 깨져 기체가 의도하지 않은 방향으로 회전한다. ③ **collective 이동** — `collectiveLo = minMotor - min(diff)` , `collectiveHi = maxMotor - max(diff)` 로 요청 throttle을 클램프한다. 즉 **자세 차동을 온전히 유지하기 위해 collective를 위아래로 밀어낸다**. 이것이 이 함수의 핵심 설계다 — 기체는 추력이 조금 달라지는 것보다 자세 제어를 잃는 쪽이 훨씬 위험하다. ④ **합성과 클램프** — `collective + diff[i]` 를 반올림해 `[minMotor, maxMotor]` 로 제한한다. 반환 구조체는 세 값을 준다: `motor[4]` (출력), `scaled` (자세 명령이 실제로 잘렸는가 — anti-windup 판단에 사용), `collective_us` (실제 적용된 collective — 자동 착륙 프로브의 전달률 검증에 사용). `scaled` 와 collective 이동을 **구분해서 보고하는 것**이 중요하다. collective만 이동한 경우는 자세 권한이 보존되었으므로 적분을 계속해도 되지만, `scaled` 인 경우는 명령이 잘렸으므로 적분을 멈춰야 한다.

4.8 캘리브레이션

```
static bool measure_imu_bias(ICM42670 &imu, const float sign[3], float bias_out[3], float sd_out[3], float &accel_mean_out)
```

역할 — 한 IMU에서 2000샘플을 모아 자이로 bias, 축별 표준편차, 가속도 크기 평균을 구한다.

기제 — 1ms 간격으로 읽으며 `sign[]` 을 곱해 IMU1 센서축으로 정렬한다. 읽기 실패는 샘플로 세지 않고 재시도하되, `maxAttempts = 2100` 으로 무한 루프를 막는다. 분산은 $E[x^2] - (E[x])^2$ 로 구하고 `double` 로 누적해 정밀도 손실을 피한다. 음수 분산은 0으로 클램프한다(부동소수 오차 방어). 가속도는 축별이 아니라 **벡터 크기**의 평균을 낸다 — 정지 상태에서 이 값은 1g여야 한다.

```
static float guarded_accel_scale(float accel_mean_g, const char *imu_name)
```

역할 — 가속도계 gain 오차 보정 계수를 안전하게 산출.

기제 — 측정된 평균이 `[0.8, 1.2]g` 밖이면 센서 이상이나 잘못된 자세로 캘리브레이션한 것이므로, **보정하지 않고** 1.0 을 반환하며 경고를 찍는다. 범위 안이면 `1/accel_mean` 을 반환한다. 잘못된 보정이 무보정보다 위험하다는 판단이다.

```
static bool calibrate_bias()
```

역할 — 부팅 시 두 IMU를 모두 캘리브레이션. 실패하면 시동 자체를 막는다.

기제 — 최대 3회 시도한다. 매 시도마다 두 IMU를 측정하고, 6개 축 표준편차 중 **최댓값**이 `BIAS_MOVEMENT_THRESH` (1.0 dps) 이하여야 통과한다. 움직임이 감지되면 재시도하고, 3회 모두 실패하면 `calibration_ok = false` 가 되어 `start` 명령이 거부된다. 가속도 스케일은 자이로 검사를 통과한 뒤에만 적용한다 — 움직이는 중에 측정한 가속도 크기는 신뢰할 수 없기 때문이다. **두 IMU의 gain을 각각 보정하는 이유**는 두 센서의 오차가 독립이고 단일-IMU 폴백 경로가 존재하기 때문이다. 평균값 하나로 보정하면 한쪽만 쓰는 상황에서 오차가 남는다.

4.9 pid_task — 1kHz 제어 루프

`pid_task` (:1194-1686)는 이 시스템의 심장이다. Core 1에 고정되어 1ms마다 실행되며, 센서 읽기부터 PWM 출력까지 전 과정을 한 함수 안에서 수행한다. 아래는 한 tick의 전체 시퀀스를 14단계로 재구성한 것이다. **단계의 순서 자체가 안전 설계의 일부**이며, 여러 단계의 위치에 "여기 두지 않으면 어떤 사고가 나는지"가 주석으로 기록되어 있다.

초기화 (태스크 진입 시 1회)

- `esp_task_wdt_add(NULL)` — 이 태스크를 위치독에 등록.
- D항 LPF 3개(40Hz), 자이로 LPF 6개(80Hz × 2 IMU × 3축) 생성.
- `FreezeMon` 2개, `LandDetector`, 프로브 설정 복사본 준비.
- `wasLocked = true` 로 시작 — 부팅 직후는 잠금 상태.

매 tick 시퀀스

#	단계	수행 내용과 설계 의도
1	주기 대기과 위치독 급이	<code>vTaskDelayUntil</code> 로 정확히 1ms 대기(busy-wait 없음) 후 <code>esp_task_wdt_reset()</code> . 큰 지연 후 밀린 tick을 연속 실행하지 않도록 <code>wake</code> 를 현재 시각으로 재설정한다. catch-up 실행은 stale 센서 읽기와 D항 spike를 만든다.

#	단계	수행 내용과 설계 의도
2	대기 중인 안전 요청 반영	<code>applyPendingSafetyRequest()</code> . Core 0의 arm/disarm 요청이 여기서 실제 <code>safety_lock</code> 전이가 된다.
3	루프 주파수 측정과 dt 계산	1초마다 <code>pidLoopHz</code> 를 갱신한다. <code>realDt</code> 가 0.2ms 미만이거나 10ms 초과면 공칭값 1ms로 대체한다 — 지연 후 catch-up에서 비정상적으로 작은 dt가 D항을 폭발시키는 것을 막는다.
4	센서 읽기와 freeze 판정	두 IMU를 SPI로 읽고 <code>checkFreeze</code> 를 각각 돌린다. freeze는 <code>fault_imu*</code> 에 래치되고 <code>imu*_frozen_now</code> 에는 현재 상태가 들어간다.
5	raw 스트림 게시	<code>raw_stream_enabled && connectionEstablished</code> 일 때만 링버퍼에 push한다. OFF일 때 hot path는 bool 검사 하나뿐 이며, ON일 때만 <code>micros()</code> 와 27바이트 복사를 한다. 부호·스케일·bias-LPF·축 변환을 전혀 거치지 않은 레지스터 원값을 보낸다.
6	부호 정렬·bias 보정·LPF	IMU2에 <code>IMU2_SIGN</code> 을 곱하고 각 IMU의 bias를 뺀다. 그 뒤 80Hz 소프트웨어 LPF를 통과시킨다. 제어와 불일치 판정 모두 필터된 값을 쓴다 — 진동으로 순간 차이가 15dps를 넘어 비행 중 disagree 컷이 나는 것을 막기 위해서다. 가속도는 각 IMU의 개별 스케일만 적용하고 LPF는 걸지 않는다(하드웨어 LPF 25Hz로 충분).
7	IMU별 텔레메트리 게시	LPF 후 자이로와 LPF 없는 가속도를 같은 1kHz 프레임에서 기체축으로 변환해 게시한다. 변환은 락 밖에서, 복사는 락 안에서.
8	이중화 융합	네 갈래다. 둘 다 정상·불일치 없음 → 단순 평균, <code>active_imus=2</code> . 한쪽만 정상 → 그쪽만 사용, <code>active_imus=1</code> . 둘 다 고장 또는 중재 불가 → <code>active_imus=0</code> , <code>safety_lock=true</code> , 적분·목표·LPF 전부 리셋, <code>stopMotors()</code> , <code>continue</code> . 불일치는 150ms 지속돼야 <code>fault_disagree</code> 로 확정된다.
9	기체축 변환	<code>bodyGx = fg[1]</code> , <code>bodyGy = -fg[0]</code> , <code>bodyGz = -fg[2]</code> 와 대응하는 가속도 변환. 주석에 따르면 roll/pitch 자이로 부호는 검증된 가속도 변환에 맞춰 수정된 것이며, 이전의 반전된 부호가 추정기 지연과 rate 루프 역감쇠 를 일으켰다.
10	상보필터 자세 추정	자이로를 적분(<code>angle += rate × realDt</code>)하고, 가속도로부터 <code>atan2</code> 로 절대 각도를 구해 적분 alpha로 섞는다. yaw는 가속도로 관측할 수 없으므로 적분만 하되, <code>YAW_DEADZONE</code> (0.3dps) 미만은 무시해 정지 시 드리프트를 억제한다.
11	자기계 yaw 융합 (250Hz)	<code>magFusionCnt</code> 독립 분주기로 정확히 250Hz를 유지한다. outerCnt를 쓰지 않는 이유 는 그것이 safety-lock 경로에서 매 tick 리셋되어 <code>K_MAG</code> 의 시정수($\tau \approx 4s$)가 깨지기 때문이다. 스냅샷의 신선도(100ms)와 크기(0이 아님)를 확인하고, throttle 간섭 보정을 뺀 뒤 틸트 보상 heading을 구해 <code>angleZ</code> 에 미세 보정을 더한다.
12	안전 검사와 failsafe 진입	과도 기율기 검사 → resume 요청 처리 → 잠금 해제 첫 tick 정리 → 호버 추정기 갱신 → RC 타임아웃 판정 → 하강 중 목표 덮어쓰기. 이 순서가 매우 중요하다 (아래 별도 설명).
13	캐스케이드 PID	바깥 250Hz: 각도 오차 × <code>Kp_Angle</code> → 목표 각속도(클램프). 안쪽 1kHz: 각속도 오차 PID. D항은 setpoint가 아니라 측정 자이로의 미분 에 걸여 setpoint 변화 kick을 제거한다.
14	믹싱·anti-windup·출력	<code>mixAndDesaturate</code> → 프로브 전달률 기록 → 조건부 적분 → PWM 4채널 출력 → 텔레메트리 미리 갱신.

12단계 내부의 순서 제약 — 주석이 기록한 세 가지 사고

① 잠금 해제 정리 블록은 RC 타임아웃 가드보다 먼저 와야 한다

`wasLocked && !safety_lock` 블록은 `fs_phase = FS_NONE` 을 포함한다. 이 블록을 RC 타임아웃 블록 뒤에 두면, 같은 tick에서 새로 진입한 `FS_DESCENDING` 을 `FS_NONE` 으로 덮어써 자동착륙이 즉시 취소된다. (:1443-1444)

② RC 타임아웃 진입 블록 전체가 `fs_phase` 가드 안에 있어야 한다

가드가 없으면 하강 내내 `rcTimedOut` 이 참이므로 이 블록이 1kHz로 재실행된다. 그러면 `Serial.println` 이 TX 버퍼를 포화시켜 `pid_task` 를 블로킹하고, **500ms** 태스크 위치독이 비행 중 재부팅을 일으킨다. 각 진입 로그에 "한 번만"이라는 주석이 붙어 있는 이유다. (:1475-1478)

③ `hover_valid` 는 절대 만료시키지 않는다

`false` 는 "RC 끊김 시 즉시 컷"을 뜻한다. 공중에서 freshness 만료로 `false` 가 되면 곧바로 추락한다. **낮은 추정치가 덜 위험하다**는 판단으로, 한 번 유효해진 추정치는 비행 중 조건이 잠시 깨져도 보존된다. (:657-659)

RC 타임아웃 시 3분기 판정

RC가 500ms 이상 끊기면 `fault_rc` 를 세우고 다음 셋 중 하나를 택한다.

조건	동작	근거
<code>!hover_valid</code>	즉시 컷	하강 목표를 계산할 기준이 없다. 시간 기반 하강을 할 수 없으므로 모터를 끈다.
<code>base_throttle ≤ 1150</code>	즉시 컷	실측 호버는 약 1340μs다. 무장 직후 spool-up 로그에서 1100~1190 구간이 전체의 9.4%였으므로, 1150 이하를 보수적으로 "지상"으로 본다. 이것은 고도 센서가 아니라 절대 하한 휴리스틱이다.
그 외	자동착륙 진입	저장된 roll/pitch 트림을 유지하며 <code>hover_est - 60μs</code> 로 하강한다. 진입 시 각 hold yaw-프로브 딥 크기를 확정한다.

자동착륙은 착지를 감지하지 않는다

하강 블록 안에 `const bool landed = false;` 가 명시적으로 박혀 있다 (:1548). 능동 프로브는 계속 실제 스로틀에 답을 넣고 응답을 기록하지만, 그 결과는 `(void)probe_says_landed;` 로 버려진다. 실제 컷은 `FS_MAX_MS` (3000ms) 타임아웃 백스톱이 한다. 따라서 **접지 후에도 남은 시간만큼 프로펠러가 돌고, 시간이 끝날 때 아직 공중일 수도 있다.** 프로브를 판정에서 뺀 경위는 4.9절 말미와 5.1절 참조.

조건부 적분 anti-windup

```
if (throttle > 1100 && !mix.scaled) {  
    iTerm += Ki * error * realDt;          // ±50μs로 클램프  
} else if (throttle <= 1100) {  
    iTermRoll = iTermPitch = iTermYaw = 0; // 지상에서는 완전 리셋  
}
```

두 조건이 각각 다른 문제를 막는다. `throttle > 1100` 은 지상에서 적분이 쌓여 이륙 순간 튀어나가는 것을, `!mix.scaled` 는 믹서가 명령을 자른 상태에서 적분이 폭주하는 것을 막는다. **collective만 이동한 경우는 적분을 계속한다** — 자세 권한이 보존되었기 때문이다.

적분을 **모터 출력 기여(μs) 단위로** 누적하는 것도 의도적이다. 이전 구현은 각도 오차합을 $\pm 200\text{deg}$ 로 클램프했는데, 기본 Ki에서 그 최대 기여가 $1\mu\text{s}$ 에 불과해 적분이 사실상 없는 것과 같았다.

yaw 적분이 rate 모드에서도 도는 이유

`iTermYaw` 는 heading hold 여부와 무관하게 누적된다. 주석 (:1659-1663)의 근거는 명확하다: 안쪽 루프의 임무는 "목표 각속도 추종"이고, P 단독으로는 모터 토크 불균형 같은 일정 외란에서 정상상태 각속도 오차가 남는다(2026-07-27 실측 최대 +16.6dps). 그러면 `YAW_HOLD_SETTLE_DPS` (10dps) 아래로 내려가지 못해 **자동 heading 잠금이 영영 걸리지 않는다**.

4.10 udp_task — 통신 태스크

`udp_task` (:1997-2268)는 Core 0에서 돌며 매 반복마다 다음을 수행한다. 우선순위가 1로 `pid_task` (2)보다 낮아, 제어 루프를 절대 선점하지 않는다.

1. **UDP 패킷 수신·파싱** — `parsePacket()` 이 0이 아니면 발신자 주소·포트를 저장하고 `connectionEstablished = true`. 최대 255바이트를 읽어 널 종단 후 명령 디스패치.
2. **자기계 풀링** — `mag_enabled || mag_calibrating` 일 때만 `sampleMagnetometer(now)`.
3. **MSP UART 풀링** — `pollMspSensor(now)`, 반복당 최대 64바이트.
4. **바이너리 스트림 송신** — 10ms마다 조건을 검사해 `ZIMU` 또는 `ZCAL` 을 최대 하나만 보낸다.
5. **ASCII 텔레메트리 송신** — 바이너리를 보내지 않은 반복에서만, 50ms 경과 시 `sendTelemetry()`.
6. `vTaskDelay(1)` 로 양보.

송신 경로가 서로를 양보하는 구조

한 반복에서 `ZIMU` 와 `ZCAL` 은 둘 중 하나만, 그리고 바이너리를 보냈으면 ASCII는 다음 반복으로 미룬다. WiFi 스택에 한 번에 여러 데이터그램을 밀어넣어 지연이 튀는 것을 막으면서, ASCII 텔레메트리의 20Hz 주기는 유지된다.

명령 디스패치 순서와 그 함정

명령은 `strcmp` 체인으로 분기하는데, **접두사가 겹치는 명령들의 순서가 정확해야 한다**.

```
rcr - 먼저 검사 (뒤에 공백/탭 필수)  ← "rc"보다 앞
rc  - 그 다음 (뒤에 공백/탭 필수)
start / stop / resume / gains      ← 완전 일치(strcmp)
th / yaw
magcal - 먼저 (6자)                  1
magc   - 그 다음 (4자)              1  길이 긴 것부터
mag    - 마지막 (3자)              1
raw / trim
그 외 → handleGainCommand(buf)     ← 2글자 계인 명령 fallback
```

만약 `mag` 를 `magcal` 보다 먼저 검사하면 `magcal 1` 이 `mag` 로 잡혀 `parseIntStrict("cal 1")` 이 실패하며 조용히 무시된다. `rc` / `rcr` 에 공백 확인을 붙인 것도 같은 이유다.

4.11 명령 파서 함수군

```
static char *trimCommand(char *s)
```

역할 — 앞뒤 공백·탭·CR·LF 제거.

기제 — 앞은 포인터를 전진시키고, 뒤는 널 문자를 써넣는다. 새 버퍼를 만들지 않는 in-place 방식이라 heap 할당이 없다.

```
static bool parseFloatStrict(const char *text, float &out)
```

역할 — 문자열 전체가 유효한 실수일 때만 파싱 성공.

기제 — `strtof` 후 세 가지를 검사한다. ① 한 글자도 소비하지 않았으면 실패, ② `isfinite` 가 거짓이면 실패 (NaN·Inf 입력 차단), ③ 후행 공백을 건너뛴 뒤에도 문자가 남아 있으면 실패. `atof` 와 달리 "12abc" 를 12로 받아들이지 않는다.

NaN 차단이 특히 중요하다 — NaN 목표각이 들어오면 모든 비교가 거짓이 되어 클램프를 통과하고 PID 전체를 오염시킨다.

```
static bool parseIntStrict(const char *text, long &out)
```

역할 — 문자열 전체가 10진 정수일 때만 성공.

기제 — `strtoul` + 동일한 전량 소비 검사.

```
static bool setRcTargets(float x, float y, float z, bool hasYaw)
```

역할 — 목표 자세를 갱신하고 RC 위치독을 급인다.

기제 — 유효성 검사 후, **자동착륙 하강 중이면 위치독만 갱신하고 목표는 건드리지 않은 채 true를 반환한다**. 이것이 "링크가 돌아와도 하강이 자동으로 끝나지 않는다"는 정책의 구현이다 — RC가 살아 있음은 인정하되 조종 권한은 돌려주지 않고, 명시적 `resume` 을 요구한다. 정상 시에는 `constrain(x + trim_roll, ±30°)` 로 **트림을 더한 뒤 클램프**한다. 순서가 반대면 총합이 30°를 넘을 수 있다.

```
static void handleRcCommand(char *buf) / handleRcrCommand(char *buf)
```

역할 — `rc` (yaw 각도)와 `rcr` (yaw 각속도) 명령을 파싱. 주력 지상국은 `rcr` 만 쓴다.

기제 — `strtok_r` 로 토큰을 나눈 뒤 다음을 수행한다. ① **형식 검증** — 4토큰 표준형 `rc <seq> <r> <p> <y>`. `rc` 는 벤치 수동 테스트용 2.3토큰 형식도 받지만 `rcr` 은 4토큰만 받는다. 여분 필드가 있으면 거부한다. ② **시퀀스 번호 처리** — 부호 (+ / -)로 시작하면 거부하고, `strtoul` + `ERANGE` 검사로 `uint32` 범위를 넘으면 거부한다. 그 뒤 `advance = (int32_t)(seq - lastRcSeq)` 를 계산한다. `advance ≤ 0` 이면 지연 도착·중복 패킷이므로 `rcDroppedPkts` 를 올리고 **폐기**한다. `advance > 1` 이면 그 사이 `advance-1` 개가 유실된 것으로 집계한다. **부호 있는 캐스팅 덕분에 uint32 wrap도 정상 처리된다** — `0xFFFFFFFF` 다음 0이 와도 `advance = 1` 로 계산된다. ③ **목표 적용** — `rcr` 은 roll/pitch만 `setRcTargets(..., hasYaw=false)` 로 넘겨 yaw 각도를 건드리지 않고, yaw는 `targetYawRate` 에 `±180dps`로 클램프해 넣는다.

```
static void handleGainCommand(const char *buf)
```

역할 — 2글자 게인 명령 22종을 처리.

기제 — 값이 `[0, 100]` 범위이고 하강 중이 아닐 때만 적용한다. 두 계열의 명령어가 공존한다: 캐스케이드 전용 (`rp/ri/rd`, `ap/ar/at/ay`, `yp/yi/yd`)과 `tune_pid.py` 호환 (`pa/ia/da`, `pr/ir/dr`, `pp/ip/dp`, `py/iy/dy`). 후자의 P/I/D는 안쪽 rate PID에 대응된다.

상태 변경 명령의 안전 게이트

명령	거부 조건과 이유
<code>start</code>	① 하강 중 → 거부. ② 이미 무장 상태 → 무시 (지상국이 <code>start</code> 를 여러 번 재전송하므로, 지연 도착한 중복 <code>start</code> 가 비행 중 <code>fault</code> 래치를 지우고 스로틀 창을 리셋하는 것을 막는다). ③ 캘리브레이션 실패·과도 기울기·양쪽 IMU freeze·불일치·자기계 보정 중 → 거부하고 각 플래그를 로그로 출력. 통과 시 Core 0 소유 상태(<code>fault</code> 플래그, 시퀀스, 스로틀 창, 목표각, <code>angleZ=0</code>)를 초기화하고 <code>requestSafetyArm()</code> .
<code>stop</code>	무조건 수락. <code>requestSafetyDisarm()</code> + <code>base_throttle=1000</code> .
<code>resume</code>	<code>resumeRefusalReason()</code> 의 6개 조건을 모두 통과해야 함(4.5절).
<code>yaw 1</code>	무장 중이면 거부 . 비행 중 임의로 커먼 지상국 setpoint와 어긋난 채 최대 권한 슬램이 날 수 있다. <code>yaw 0</code> 은 언제나 허용.
<code>magcal 1</code>	무장 중이면 거부(블로킹 I2C 초기화 포함).
<code>mag 1</code>	보정 중이면 거부. 초기화되지 않았는데 무장 중이면 거부. 이 때문에 지상국은 start보다 먼저 <code>mag</code> 명령을 보낸다.
<code>th <v></code>	1000~1900으로 클램프하고 <code>min/max_throttle</code> 창을 $\pm 150\mu\text{s}$ 로 재설정.
<code>trim <r></code> <code><p></code>	$\pm 10^\circ$ 로 클램프. 하강 중 거부. 절대값 명령이라 재전송이 안전하다.
<code>raw 0 1</code>	OFF→ON 전환 시 링버퍼를 비우고 (<code>tail=head</code>) 생산자·소비자 시간 기준을 무효화한다. 이전 세션의 낡은 샘플이 새 캡처에 섞이는 것을 막는다.
<code>gains</code>	12개 계인을 <code>GAINS,...</code> 형식으로 회신.

4.12 텔레메트리 송신 함수군

```
static void sendTelemetry()
```

역할 — 65개 필드를 십표 구분 ASCII로 20Hz 송신.

기제 — 먼저 모든 값을 **지역 스냅샷으로 복사**한 뒤 `udp.printf` 한 번에 포맷한다. 락이 필요한 값(IMU별 샘플, 자기계 텔레메트리)은 전용 `read` 함수로 가져온다. `criticalFault` 는 `active_imus==0 || fault_attitude || !calibration_ok` 의 합성값으로, 하나의 필드로 "지금 치명적인가"를 알려준다. **필드는 append-only가 계약이다** — 중간 삽입이나 순서 변경은 모든 구버전 파서를 깨뜨린다. 전체 명세는 부록 13.1 참조.

```
static bool sendImuRawBatch()
```

역할 — 링버퍼의 샘플을 최대 50개씩 `ZIMU` 데이터그램으로 전송.

기제 — 핵심은 **시각 복원**이다. 각 샘플은 직전 샘플과의 `dt_us` 만 갖고 있으므로, 소비자는 첫 샘플의 절대 시각 (`first_t_us`)에서 출발해 `dt` 를 누적한다. `rawConsumerTimeValid` 가 거짓인 첫 배치에서만 링버퍼의 `first_t_us` 를 읽고, 이후에는 `rawConsumerLastUs + dt_us` 로 이어간다. 배치의 첫 샘플 시각이 헤더의 `base_t_us` 가 되어 PC가 같은 방식으로 복원한다. 생산자의 `dropped` 카운터도 헤더에 실어, PC가 "무선 유실"과 "펌웨어 큐 넘침"을 구분할 수 있게 한다.

```
static bool sendImuCalibration()
```

역할 — 1초마다 ZCAL 60바이트를 보낸다.

기제 — 자이로 bias 2세트, 가속도 스케일 2개, 공통 스케일 상수 2개, IMU2 부호 3개를 담는다. 1Hz로 반복 전송하는 이유는 **UDP라 유실될 수 있고, 캡처를 중간부터 시작해도 보정값을 얻을 수 있어야** 하기 때문이다. PC 디코더는 첫 ZCAL을 만나기 전의 샘플에도 소급 적용한다.

```
static void pollMspSensor(uint32_t nowMs)
```

역할 — 3901-L0X UART를 논블로킹으로 폴링해 거리·광류를 갱신.

기제 — `available()` 이 있는 만큼만, 반복당 최대 64바이트까지 `mspSensorParseByte` 에 넘긴다. 실제 유량은 $20\text{Hz} \times \text{약 } 15\text{B} = 300\text{B/s}$ 라 이 상한에 닿지 않지만, **BMM350의 블로킹 I2C가 RC 수신 큐를 넘치게 했던 전례** 때문에 이 경로는 절대 블로킹하지 않도록 설계했다. 신선도 관리가 특히 중요하다: 500ms 넘게 프레임이 없으면 `quality` 를 -1 로 내린다. **거리값 자체는 건드리지 않는다** — 모듈이 범위 밖에서 음수를 쓰므로 센티넬로 쓸 수 없고, 너무 가까울 때와 너무 멀 때가 같은 음수라 부호만으로는 구분이 불가능하다. 따라서 신선도 판단은 반드시 `*_Quality == -1` 로 해야 한다.

4.13 setup과 loop

```
void setup()
```

역할 — 하드웨어 초기화와 두 태스크 생성. 실패하면 무한 루프로 정지한다.

기제 — 순서 자체가 안전 설계다. ① **CS 핀부터** — 두 IMU의 CS를 OUTPUT/HIGH로 먼저 설정한다. 플로팅 CS는 SPI 버스 경합을 일으킨다. ② **WiFi SoftAP** — `setSleep(false)` 로 절전을 끄고 TX 파워를 19.5dBm으로 최대화한다. 절전이 켜져 있으면 RC 지연이 수백 ms까지 튜다. ③ **UART1** — TX 핀을 -1 로 두어 GPIO 하나만 점유한다. ④ **ESC attach 후 즉시 stopMotors()** — PWM이 살아나는 순간 아이들 신호를 보장한다. attach 실패는 `while(1)` 로 정지한다. ⑤ **IMU 시작** — `begin` → `startAccel/startGyro(1600Hz)` → `setLowPassFilters()` . 하나라도 실패하면 정지. ⑥ **캘리브레이션** — 결과를 `calibration_ok` 에 저장. 실패해도 부팅은 계속하지만 `start` 가 거부된다. ⑦ **위치독 설정** — `idle_core_mask = 0` 으로 idle 태스크는 감시하지 않고, `trigger_panic = true` 로 timeout 시 재부팅을 강제한다. `reconfigure` 가 실패하면 `init` 을 시도한다. ⑧ **태스크 생성** — pid를 Core 1/우선순위 2, udp를 Core 0/우선순위 1로 고정한다. 생성 실패 시 `safety_lock` 을 세우고 정지한다.

```
class ICM42670WithLPF : public ICM42670
```

역할 — 라이브러리가 노출하지 않는 하드웨어 LPF를 켜기 위한 상속 클래스.

기제 — `startGyro / startAccel` 은 ODR과 FSR만 설정하고 UI 필터 대역폭은 칩 전원 기본값(bypass)으로 둔다. 그러면 **프로펠러 진동이 필터 없이 PID에 들어온다**. 이 클래스는 상속으로 `protected` 멤버 `icm_driver` 에 접근해 `inv_imu_set_gyro_ln_bw(121Hz)` 와 `inv_imu_set_accel_ln_bw(25Hz)` 를 직접 호출한다.

```
void loop()
```

역할 — 평시에는 아무것도 하지 않는다.

기제 — `delay(1000)` 만 수행한다. 주석이 이유를 밝힌다: **UDP 객체는 udp_task 한 곳에서만 접근해 cross-core race를 피한다**. `WIFI_LATENCY_DEBUG` 빌드에서만 지연 이벤트 링버퍼를 비우고 2초마다 요약 통계를 출력한다.

선택적 지연 계측 시스템

#if WIFI_LATENCY_DEBUG 로 감싸인 코드 블록들(:90-134 , :511-561 등)은 UDP 루프 간격, 파싱 시간, 송신 시간, PID 주기를 임계값 초과 시에만 128칸 링버퍼에 기록한다. **기본 빌드에서는 완전히 컴파일 제외**되므로 실비행 성능에 영향이 없다. 30ms/100ms/300ms 구간별 초과 횟수를 누적해, WiFi 스택이 제어 루프를 방해하는지 정량적으로 판단할 수 있게 한다.

5. 주력 펌웨어 동반 헤더 4종

이 네 헤더는 **Arduino** 런타임에 의존하지 않는 순수 함수와 자료구조만 담고 있다. 그래서

`tools/native_tests/` 가 PC에서 `g++` 로 직접 컴파일해 수백 개의 단위검사를 돌릴 수 있다. 이 저장소의 검증 전략 전체가 이 분리에 의존한다.

5.1 failsafe_land.h — 자동착륙과 호버 추정

열거형 2종 (텔레메트리 wire 계약)

열거형	값
<code>FailsafePhase</code> 텔레메트리 필드 36	<code>0 FS_NONE</code> · <code>1 FS_DESCENDING</code> · <code>2 FS_CUT_LANDED</code> · <code>3 FS_CUT_TIMEOUT</code> · <code>4 FS_CUT_ABORT</code>
<code>FailsafeProbeState</code> 텔레메트리 필드 41	<code>0 WAIT</code> · <code>1 DIP</code> · <code>2 EVALUATE</code> · <code>3 UNAVAILABLE</code> · <code>4 BLOCKED</code>
<code>ResumeRefusalReason</code>	<code>ALLOWED</code> · <code>PHASE</code> · <code>RC</code> · <code>TILT</code> · <code>IMU</code> · <code>HOVER</code> · <code>CUMULATIVE</code>

```
struct HoverThrottleEstimator { estimate_us, initialized, timing_initialized, valid, last_update_ms, eligible_ms }
```

역할 — 조종자가 호버를 유지하는 동안의 collective를 관찰해, RC 상실 시 쓸 하강 기준값을 만든다.

```
static inline void updateHoverThrottleEstimator(HoverThrottleEstimator &, bool sample_eligible, int throttle_us, uint32_t now_ms, float dt_s, float tau_s, uint32_t valid_ms)
```

역할 — 매 tick 호출되어 호버 추정치를 갱신한다.

기제 — 호출자(`pid_task`)가 다음 5개 조건을 모두 만족할 때만 `sample_eligible = true` 를 준다: 무장 상태, `fs_phase == FS_NONE`, $|\text{roll}|, |\text{pitch}| < 10^\circ$, $||\mathbf{a}| - 1| \leq 0.05g$, `throttle > 1150\mu s`. 즉 **자세도 수직 가속도도 정상상태인 구간만** 호버 후보로 본다. 부적합 구간에서는 `return` 으로 빠져나가 **누적과 LPF를 함께 멈추되, 이미 모은 시간과 추정치는 보존한다**. 적합 구간에서는 첫 샘플을 그대로 대입하고, 이후에는 $\alpha = dt / (\tau + dt)$ 인 1차 LPF로 천천히 추적한다($\tau=3\text{초}$). 적합 시간이 `valid_ms` (1500ms) 이상 누적되면 `valid = true` 가 되며, **이 플래그는 다시 내려가지 않는다**. `eligible_ms` 누적에는 `UINT32_MAX` 포화 처리가 들어 있어 장시간 비행에서 오버플로가 나지 않는다.

```
static inline int failsafeProbeDipUs(float hover_estimate_us, float dip_frac, int min_us, int upper_exclusive_us, bool &clamped)
```

역할 — 기체 질량에 비례하는 프로브 딥 크기를 계산하고 클램프 여부를 보고.

기제 — $(\text{hover_est} - 1000) \times \text{dip_frac}$ 을 반올림한다. 아이들 1000 μs 를 뺀 **유효 collective**에 비례시킴으로, 질량이 달라져도 같은 주력 비율의 딥이 나온다. 결과를 `[min_us, upper_exclusive_us)` 로 제한하고 `clamped` 출력 인자로 클램프 발생을 알린다. 입력이 비유한하거나 범위가 역전되면 `min_us` 를 반환하고 `clamped = true` .

이 기체에서는 답을 더 키울 수 없다 — R13 회귀 시험이 잡아낸 한계

전달 가능한 답의 상한은 $(\text{hover_est} - \text{FS_DESCENT_DELTA_US}) - \text{CTRL_MARGIN} - 1050$ 이다. $\text{hover_est} = 1290$ 에서 그 값은 $30\mu\text{s}$ 인데 현재 답이 이미 $34\mu\text{s}$ 다. 더 키우면 믹서가 collective를 밀어 올려 전달률이 80% 아래로 떨어지고 FS_PROBE_BLOCKED 가 된다(2배로 키우면 전달률 44%). 답을 늘리려면 CTRL_MARGIN 이나 1050 하한을 먼저 손봐야 하는데 **둘 다 안전 관련**이라 별도 검토가 필요하다. 2026-08-01에 "답을 2배로 올려 공중 응답만 키우자"는 시도가 이 계산으로 기각됐다.

```
static inline bool updateLandDetector(LandDetector &, float accel_magnitude_g, uint32_t elapsed_ms, int base_throttle_us, const LandProbeConfig &)
```

역할 — 능동 스로틀 프로브 상태기계를 구동하고 착지 후보 여부를 반환. **현재 반환값은 제어에 쓰이지 않는다.**

기제 — 원리는 이렇다. 공중에 있으면 스로틀을 잠깐 줄일 때 기체가 **가속도로 반응**한다(낙하 방향 가속). 접지해 있으면 지면이 받치므로 **반응이 없다**. 그 차이로 착지를 판정하려는 것이다. 상태 전이: **WAIT** → 다음 프로브 시각이 되면 현재 LPF 값을 pre_dip_g 로 저장하고 **DIP**로. 단, 답을 넣으면 스로틀이 $1000\mu\text{s}$ 아래로 갈 상황이면 **UNAVAILABLE**로 고정된다. **DIP** → 답 시작 후 sample_delay_ms (30ms)가 지난 뒤부터만 최소값을 갱신한다(초기 과도 응답 배제).

dip_ms (120ms)가 지나면 $\text{response} = \text{pre_dip_g} - \text{dip_min_g}$ 를 확정한다. 전달률이 부족했으면 **BLOCKED**(카운터를 올리지도 내리지도 않음), 아니면 **EVALUATE**로 가서 응답이 임계(0.03g)를 넘으면 무반응 카운트를 0으로, 아니면 +1. **EVALUATE/BLOCKED** → 다음 tick에 **WAIT**로. 판정과 전환을 별도 tick으로 유지해 20Hz 텔레메트리가 상태를 놓치지 않게 한다. LPF는 min_descend 게이트보다 **먼저 항상** 갱신한다 — 프로브가 시작될 때 필터가 이미 정착해 있어야 pre_dip_g 가 의미를 갖는다. 반환 조건은 $\text{no_response_count} \geq \text{confirm_n}$ **그리고** $|\text{filt} - 1.0| \leq \text{tol}$ 이다. 후자는 "지면에 정지하면 반드시 1g"라는 사실을 **필요조건**으로만 추가한 것이며, 1g만으로는 true가 되지 않는다.

프로브가 판정에서 빠진 경위 — 철회된 근거와 유효한 관측

코드 주석이 이 실패를 이례적으로 정직하게 기록하고 있다.

- **철회된 관측**: "2026-08-01 공중 오판 9회 중 4회"는 **20Hz 단일 샘플을 자유낙하로 오독한** 것이었다. 그 하강은 실제로는 지면에서 났고, 1kHz 기록과 조종자 확인이 접지를 지지한다. 즉 공중 오판은 일어나지 않았다. 이에 따라 "지면 응답이 공중보다 큼 / 판별 방향 반대 / 오분류 9/18"도 모두 근거를 잃었다.
- **유효한 실패 관측**: log 015448에서 임계를 0.03으로 내린 뒤 **접지 응답 10회가 전부 임계 위가** 되어 착지를 확정할 수 없었다.
- **결정적 관측**: 답이 없어도 호버 중 120ms 창 의 $|\text{accel}|$ 낙폭이 p95 **0.0904g**로 임계 0.06g보다 크다. **교란만으로 프로브 신호 전체 크기가 나온다**. 연속 판정 횟수를 늘려도 판별식이 갖지 못한 정보를 만들 수는 없다.

결론: 프로브는 제어에서 제외되고 텔레메트리 기록 전용이 되었다. 근본 대책은 거리계로 착지를 **측정**하는 방향이며 docs/failsafe_land_research.md 에 정리되어 있다.

```
static inline void recordLandDetectorProbeDelivery(LandDetector &, uint32_t elapsed_ms, bool delivery_valid, const LandProbeConfig &)
```

역할 — 믹서가 요청한 답을 실제로 전달했는지를 검출기에 되먹인다.

기제 — mixAndDesaturate **뒤에** 호출된다. 표본 채집 구간(30~120ms)의 한 tick이라도 전달률이 80% 미만이면 $\text{probe_delivery_valid} = \text{false}$ 를 래치한다. 자세 차동이 collective 하한을 밀어 올려 답이 잘리면, 그 부분 답의 최소값을 신뢰하면 안 되기 때문이다. 이 되먹임이 없으면 검출기가 **공중을 지면으로 오판**할 수 있다.

```
static inline int failsafeDescentThrottle(int entry_throttle, int descent_delta_us)
```

역할 — 하강 목표 스로틀 계산.

기제 — `entry - delta`, 단 1000 미만은 1000으로.

```
static inline FailsafePhase failsafeStep(bool landed, uint32_t elapsed_ms, uint32_t max_ms)
```

역할 — 자동착륙의 종료 조건 판정.

기제 — 3줄이다. `landed` 면 `FS_CUT_LANDED`, 시간 초과면 `FS_CUT_TIMEOUT`, 아니면 계속 `FS_DESCENDING`. **착지가 타임아웃보다 우선한다.** 현재 호출자가 `landed=false` 를 고정으로 넘기므로 실제로는 타임아웃 경로만 동작한다.

FS_MAX_MS = 3000 의 실측 근거

프로브가 판정에서 빠진 뒤 이 값은 **하강 시간 예산**이 되었다. 코드 주석에 2026-08-01 실측이 남아 있다. 접지 충격 임펄스를 적분해 접지 속도를 재고 등가속 모델 $v = a \cdot t$, $h = \frac{1}{2}a \cdot t^2$ 로 역산한 결과다.

로그	t	v	a	역산 고도 h
023021	0.80 s	0.76 m/s	0.95 m/s ²	0.30 m
023226	0.82 s	1.30 m/s	1.59 m/s ²	0.53 m
023904	1.76 s	2.11 m/s	1.20 m/s ²	1.86 m ← 환경상 최고 고도

평균 $a \approx 1.24 \text{ m/s}^2$ (추력 부족 60/290 = 21% = 2.06 m/s²에서 항력만큼 감소). 이 평균을 일정하다고 가정한 외삽은 2.0s→2.49m, 2.5s→3.89m, 3.0s→5.60m, 3.5s→7.62m다. **다만 주석이 명시하듯 3.0초에서 실제로 5.60m를 내려온다는 보장은 없다** — 이는 실측 고도 한계가 아니라 외삽이다.

5.2 yaw_command.h — yaw 각속도 명령과 heading 잠금

이 헤더는 39행에 불과하지만 조종감을 좌우하는 핵심 로직을 담는다. yaw 스틱은 **절대 각도가 아니라 각속도 명령**이며, 스틱을 놓으면 **회전이 멈춘 그 방향을 자동으로 잠근다**.

```
static inline YawOuter updateYawOuter(float target_yaw_rate_dps, float body_gz_dps, float angle_z_deg,
float prev_target_angle_deg, bool prev_hold, bool override_hold, float rate_deadzone_dps, float settle_dps)
```

역할 — 매 tick yaw 모드(rate/hold)와 heading setpoint를 결정.

기제 — 두 줄로 요약된다.

```
hold = override_hold || (stick_centered && (prev_hold || settled))
target_angle = hold ? prev_target_angle : angle_z // 슬레이빙
```

`stick_centered` 는 `|명령 각속도| < 3dps`, `settled` 는 `|실제 각속도| < 10dps` 다. **세 가지 설계 포인트가 있다.** ① **hold가 아닌 동안 setpoint를 매 tick 현재 heading으로 슬레이빙한다.** 따라서 setpoint가 stale해질 수 없고, rate→hold 전환은 **정의상 bumpless**다. 큰 heading 드리프트 상태에서 잠금에 들어가도 슬램이 발생하지 않는다. ② **settled** 는 **진입 게이트일 뿐 해제 조건이 아니다.** 일단 잠기면 `prev_hold` 가 참이므로, 외란으로 각속도가 임계를 왕복해도 잠금이 풀리지 않는다. ③ 스틱을 밀면 `stick_centered` 가 거짓이 되어 **항상 rate 모드로 풀린다.** 조종자 입력이 언제나 우선한다.

```
static inline float yawTargetRateDps(const YawOuter &state, float target_yaw_rate_dps, float angle_z_deg, float kp_angle_yaw, float max_rate_dps)
```

역할 — 바깥 루프의 yaw 목표 각속도를 산출.

기제 — hold면 $\text{wrapDeg}(\text{target} - \text{angle_z}) \times K_p$ (각도 P 제어), rate면 명령 각속도를 그대로. 어느 쪽이든 마지막에 $\pm \text{max_rate_dps}$ 로 **하드 실링**을 건다. `wrapDeg` 를 쓰므로 $\pm 180^\circ$ 경계에서 항상 **짧은 쪽으로** 복원한다(359° 에서 1° 로 갈 때 358° 가 아니라 2° 를 돈다).

5.3 mag_yaw_fusion.h — 자기계 yaw 융합 수학

```
static inline float wrapDeg(float angle_deg)
```

역할 — 임의 각도를 $[-180, +180)$ 으로 정규화.

기제 — $\text{fmodf}(\text{angle} + 180, 360)$ 후 음수면 360을 더하고 마지막에 180을 뺀다. `fmodf` 가 음수 입력에 음수를 반환하는 C 표준 동작을 보정하는 것이 중간 단계의 목적이다.

```
static inline float tiltCompensatedMagHeadingDeg(float mag_x, float mag_y, float mag_z, float roll_deg, float pitch_deg)
```

역할 — 기울어진 상태에서도 정확한 자기 방위각을 계산.

기제 — 기체가 기울면 자기장의 수직 성분이 수평 축으로 섞여 들어와 heading이 크게 틀어진다. 이를 roll/pitch 회전을 역으로 적용해 **수평면으로 투영**한다.

```
hx = mx*cosP + my*sinR*sinP + mz*cosR*sinP
hy = my*cosR - mz*sinR
heading = atan2(-hy, hx) * 180/π
```

`atan2` 의 첫 인자에 음수를 쓰는 것은 기체측 yaw 부호 규약(CW+)에 맞추기 위해서다.

```
static inline float magYawCorrectionDeg(float yaw_deg, float mag_heading_deg, float gain)
```

역할 — 자이로 적분 yaw에 더할 보정량을 계산.

기제 — $\text{gain} \times \text{wrapDeg}(\text{mag_heading} - \text{yaw})$. $K_{\text{MAG}} = 0.001$ 이고 250Hz에서 호출되므로 시정수는 $\tau \approx 0.004/K = 4$ 초 다. 매우 느리게 당기므로 순간적인 자기 간섭 (모터 전류, 금속 근접)에 자세가 끌려가지 않으면서, 수십 초 규모의 자이로 드리프트는 제거된다.

모터 전류 간섭 보정

모터 전류는 자기장을 왜곡한다. 이 펌웨어는 body-frame ΔB 가 `base_throttle` 에 **선형 비례**한다고 근사해, hard/soft-iron 보정과 축 변환을 거친 뒤 틸트 보정 **전에** 뺀다.

```
mag.x -= mag_comp_x * (base_throttle - 1000) // μT/μs
mag.y -= mag_comp_y * (base_throttle - 1000)
mag.z -= mag_comp_z * (base_throttle - 1000)
```

2026-07-27 벤치 특성화에서 throttle 간섭이 $+3.64^\circ/100\mu s \rightarrow +0.02^\circ/100\mu s$ 로 줄었다. 2026-08-04에 soft-iron 행렬을 도입하면서 이 계수가 사는 좌표계가 바뀌었는데, 간섭은 raw 축의 물리적 ΔB 이고 보정은 선형이므로 **새 계수 = W @ 기존 계수**로 환산했다. 런타임에 `magc x y z` 명령으로 변경할 수 있고 0이면 비활성이다.

5.4 msp_sensor.h — MSPv2 센서 프레임 파서

Matek 3901-L0X가 UART로 밀어 넣는 INAV MSPv2 프레임을 바이트 단위로 해석한다. 수신 전용이며 어떤 것도 제어하지 않는다.

프레임: '\$' 'X' <dir> flags(u8) cmd(u16 LE) size(u16 LE) payload crc8
└────────── CRC 적용 구간 ─────────┘

'\$X'와 방향 바이트('<' '>' '!')는 CRC에 들어가지 않는다.

페이로드:

MSP2_SENSOR_RANGEFINDER 0x1F01 : u8 quality; i32 distanceMm (5 B)
MSP2_SENSOR_OPTIC_FLOW 0x1F02 : u8 quality; i32 motionX; i32 motionY (9 B)

static inline uint8_t mspCrc8DvbS2(uint8_t crc, uint8_t value)

역할 — DVB-S2 CRC-8 한 바이트 누적.

기제 — 다항식 0xD5 . XOR 후 8회 시프트하며 MSB가 1이면 다항식을 XOR한다. 테이블 없는 비트 단위 구현이라 RAM을 쓰지 않는다.

static inline int32_t mspReadI32(const uint8_t *p)

역할 — 리틀엔디언 4바이트를 부호 있는 32비트로.

기제 — uint32_t 로 조립한 뒤 캐스팅한다. 바이트 시프트를 직접 하므로 정렬 요구나 엔디언 가정이 없다.

static inline MspParseResult mspSensorRestart(MspSensorParser &p, uint8_t c)

역할 — 예상치 못한 바이트를 만났을 때 재동기화.

기제 — 기본은 IDLE 로 복귀하지만, 그 바이트 자체가 '\$' 이면 곧바로 HEADER_X 로 간다. 이 한 줄 덕분에 재동기화에서 한 바이트도 흘리지 않는다. 단순히 IDLE로만 돌아가면 "\$X\$X..." 같은 경우 두 번째 프레임 시작을 놓친다.

static inline MspParseResult mspSensorParseByte(MspSensorParser &p, uint8_t c)

역할 — 10상태 유한상태기계. 바이트 하나씩 먹여 프레임이 완성되면 결과를 반환.

기제 — IDLE → HEADER_X → HEADER_DIR → FLAGS → CMD_LO → CMD_HI → SIZE_LO → SIZE_HI → PAYLOAD → CRC . **주요 방어 장치 4가지.** ① **CRC 시작점** — 방향 바이트 다음부터 누적한다 (HEADER_DIR 에서 p.crc = 0). ② **oversize 처리** — 페이로드가 버퍼(16B)보다 크면 oversize 플래그를 세우고 **저장하지 않되 소비는 계속한다.** 그래야 다른 MSP 트래픽이 섞여도 프레임 경계를 잃지 않고 재동기화된다. ③ **길이 검증** — 아는 function이라도 size 가 기대값(5/9)과 다르면 **unknown으로 센다.** 조용히 잘못 해석하는 것보다 카운터가 올라가는 편이 낫다. ④ **진단 카운터** — frames_ok , crc_errors , unknown_frames , oversize_frames , last_unknown_cmd . CRC 오류나 oversize가 늘면 배선/보레이트 문제, unknown이 늘면 function 상수가 틀렸다는 뜻이다. 펌웨어는 마지막 unknown ID를 시리얼에 한 번만 출력해 "값이 안 나온다"가 아니라 **구체적 원인**으로 드러나게 한다.

거리값의 부호를 신선도 판단에 쓰면 안 된다

모듈의 작동 범위는 80~2000mm이고, distanceMm 는 범위 밖이면 음수다. 그런데 너무 가까울 때도 음수라 부호만으로는 지면이 가까운지 먼지 구분할 수 없다. 접지 판정을 하려면 직전 유효값의 추세를 봐야 하며, 신선도 판단은 반드시 quality == -1 로 해야 한다.

6. 보류 후보 펌웨어: dual_imu_flix_quat_pwm

firmware/flight/dual_imu_flix_quat_pwm/

보류

오픈소스 프로젝트 [flix](#)의 아키텍처를 ZETIN 하드웨어에 이식한 대안 비행 펌웨어다. CI 빌드 대상이지만 **실기 검증이 끝나지 않았고 주력 지상국과 호환되지 않는다**. 비행에 사용해서는 안 된다.

6.1 설계 차이 요약

항목	주력 (cascade)	보류 후보 (flix quat)
자세 표현	축별 오일러각 + 상보필터	쿼터니언 적분. 짐벌락이 없고 큰 각도에서도 특이점이 없다.
가속도 보정	매 tick 적응 alpha로 섞음	착지 상태에서만 중력 방향으로 당긴다(<code>ACC_WEIGHT=0.003</code>).
드리프트 억제	선택적 자기계 융합	비행 중 수평 가정 보정(<code>LEVEL_WEIGHT=0.0002</code>). 자기계 없음.
제어 오차	축별 각도 차	up-벡터 사이의 회전벡터 . 축 간 커플링을 자연스럽게 처리한다.
내부 단위	deg, deg/s, μ s	SI: rad, rad/s, 정규화 토크(1.0 = 모터 전 구간)
게인 단위	<code>Kp_Rate = 0.50 μs/(deg/s)</code>	<code>Kp_Rate = 0.05</code> (정규화). 서로 이식 불가
yaw 부호	CW +	CCW + (오른손 FLU)
텔레메트리	65필드	22필드
지원 명령	<code>rcr</code> , <code>gains</code> , <code>resume</code> , <code>trim</code> , <code>mag</code> , <code>raw</code> 등	<code>rc</code> 만. <code>rcr</code> · <code>gains</code> 자동착륙 미지원
RC 상실 시	조건부 자동착륙	즉시 컷만
자기계/거리계	지원	없음

6.2 수학 라이브러리 헤더

`vector.h`, `quaternion.h`, `lpf.h`, `pid.h` 는 flix 원본(Copyright © 2023 Oleg Kalachev)에서 가져왔으며 `pid.h` 만 이 프로젝트에 맞게 수정되었다.

quaternion.h (230행)

주요 멤버	역할과 기제
<code>fromAxisAngle(axis, angle)</code>	축·각으로 회전 쿼터니언 생성. $(\cos(\theta/2), \text{axis} \cdot \sin(\theta/2)/ \text{axis})$.
<code>fromRotationVector(rotation)</code>	회전벡터(크기=각도)로 생성. 영벡터면 단위 쿼터니언을 반환해 0 나눗셈을 피한다.
<code>fromEuler(euler)</code>	ZYX 오일러각 → 쿼터니언.
<code>fromBetweenVectors(u, v)</code>	두 벡터를 잇는 최소 회전. 외적을 벡터부로, <code>dot + · </code> 를 스칼라부로 두고 정규화한다.

주요 멤버	역할과 기제
<code>toEuler()</code>	쿼터니언 → 오일러각. <code>sarg</code> 가 ± 0.99999 를 넘는 짐벌락 근방을 별도 분기로 처리한다.
<code>operator*</code>	해밀턴 곱(회전 합성).
<code>conjugate(v)</code> / <code>conjugateInversed(v)</code>	$q \cdot v \cdot q^{-1}$ / $q^{-1} \cdot v \cdot q$. 벡터 회전의 두 방향.
<code>rotate(a, b)</code> / <code>rotateVector(v, q)</code>	정적 헬퍼. <code>rotate</code> 는 기본으로 정규화해 1kHz 반복 적분의 수치 드리프트를 막는다 .
<code>finite()</code> / <code>invalidate()</code>	NaN 검출과 명시적 무효화.

lpf.h — 템플릿 저역통과 필터

```
template <typename T> class LowPassFilter
```

역할 — 스칼라와 벡터 양쪽에 쓰는 1차 IIR.

기제 — `output += alpha * (input - output)`. 템플릿이므로 `float` 과 `Vector` 모두에 인스턴스화된다. `init` 플레그로 **첫 샘플은 그대로 대입**해 0에서 시작하는 과도 응답을 없앤다. `setCutoffFrequency` 는 `alpha = 1 - exp(-2π·fc·dt)` 로 정확한 지수 매핑을 쓴다 (주력의 `dt/(rc+dt)` 근사와 다르다).

pid.h — 수정된 PID 클래스

```
class PID { p, i, d, windup, dlpf, integralTerm; update(error, measurement, dt); integrate(dt); reset(); }
```

역할 — 안티와인드업이 분리된 PID 제어기.

기제 — flix 원본에서 세 가지를 바꿨다. ① 전역 시간 대신 **호출자가 dt를 넘긴다**(1kHz 태스크 기준). ② D항을 오차가 아니라 **측정값(각속도)의 미분**에 건다 — setpoint 변화 kick 제거. ③ 적분을 `update()` 에서 분리해 **별도 integrate() 호출**로 만들었다. 그래서 믹서가 포화된 tick에서는 호출을 생략하는 것만으로 조건부 적분 안티와인드업이 구현된다. `prevMeasurement` 를 `NAN` 으로 초기화하고 `isfinite` 로 검사해 첫 tick의 미분 폭주를 막는다.

6.3 추정·제어 함수군

```
static inline Vector sensorToBody(const float v[3])
```

역할 — IMU1 센서축 → 기체 FLU축.

기제 — `Vector(-v[1], v[0], v[2])`. Z축 +90° 회전에 해당한다.

```
static void applyGyro(const Vector &gyroBody, float dt)
```

역할 — 자이로로 자세를 전파.

기제 — 각속도에 LPF를 걸어 `rates` 를 갱신하고, `rates × dt` 를 회전벡터로 보아 쿼터니언을 곱한다. `Quaternion::rotate` 가 매번 정규화하므로 누적 오차가 발산하지 않는다.

```
static void applyAcc(const Vector &accBody)
```

역할 — 착지 상태에서만 중력 방향으로 자세를 당긴다.

기제 — `landed` 조건이 핵심이다: `(safety_lock || throttle ≤ 1100) && | |a| - 1 | < 0.1`. 즉 **무장 해제 또는 아 이들 스로틀이면서 가속도가 1g에 가까울 때**만 보정한다. 비행 중에는 추력 가속도가 섞여 중력 방향을 알 수 없으므로 아 예 보정하지 않는다. 보정량은 `rotationVectorBetween(accBody, up) × 0.003` 으로 매우 작다.

```
static void applyLevel()
```

역할 — 비행 중 자이로 드리프트를 억제.

기제 — "조종사가 기체를 대체로 수평으로 유지한다"는 가정에 기대, 추정 up 벡터를 월드 up 쪽으로 `0.0002` 가중치로 아주 약하게 당긴다. **가정이 틀리면 누적 편향이 생기는 trade-off**가 있지만, 자기계 없이 장시간 드리프트를 막는 저렴한 수단이다.

```
static Vector controlAttitude(const Vector &euler)
```

역할 — 자세 오차를 목표 각속도로 변환(바깥 루프).

기제 — 주력과 접근이 근본적으로 다르다. 목표 자세 쿼터니언을 만들고, **현재와 목표의 up 벡터를 각각 구해 그 사이의 회전벡터를 오차로 삼는다.**

```
upActual = rotateVector(up, attitude)
upTarget = rotateVector(up, attitudeTarget)
error     = rotationVectorBetween(upTarget, upActual)
```

이 방식은 roll-pitch 커플링을 자동으로 처리한다. yaw는 up 벡터로 표현되지 않으므로 별도로 `wrapAngle(target - euler.z) × Kp` 를 쓴다. `yaw_enabled` 가 꺼져 있으면 yaw 목표 각속도는 0이 되지만, **rate 감쇠는 계속 동작한다.**

```
static bool measure_imu_rest(...) / static bool calibrate_rest()
```

역할 — 자이로 bias 측정에 더해 **좌표계 매핑 검증**까지 수행.

기제 — 주력과 결정적 차이다. 정지 가속도를 `sensorToBody` 로 변환한 뒤 두 가지를 확인한다. ① `| |a| - 1 | > 0.3` 이면 ACCEL_SCALE이나 장착에 문제가 있다고 보고 **실패**. ② `accBody.z < 0.5` 이면 축 매핑이 하드웨어와 어긋난 것 이므로 **실패하고 시동을 막는다**. 오류 메시지가 명시적이다: "sensorToBody() axis map does not match the hardware; fix it before flying". **좌표계 실수를 부팅 시점에 잡아내는 안전장치**로, 주력 펌웨어에는 없는 기능이다.

```
static MotorMix desaturateMix(const float diffIn[4], int throttle, int minMotor, int maxMotor)
```

역할 — 주력의 `mixAndDesaturate` 와 동일한 desaturation 알고리즘.

기제 — 차이는 믹싱 부호를 **외부에서 계산해 배열로 받는다**는 점뿐이다. 호출자가 flx 부호 (`tx-ty+tz, -tx+ty+tz, -tx-ty-tz, tx+ty-tz`) × 1000 로 만든 차동을 넘긴다. 스케일-collective 이동-클램프 로직은 동일하다.

불일치 판정의 차이

주력은 제어용 80Hz LPF를 거친 자이로로 불일치를 판정하지만, 이 펌웨어는 **불일치 비교 전용으로 더 낮은 13Hz LPF(DISAGREE_LPF_ALPHA=0.08)**를 따로 둔다. 두 IMU는 보드 위 위치가 달라 진동에서 순간 차이가 15dps를 넘을 수 있으므로, 저주파 성분만 비교하겠다는 판단이다.

6.4 주력과의 비호환 지점 정리

이 펌웨어를 올리면 지상국이 정상 동작하지 않는다

- `control_dualsense.py` 는 `rcr` 만 보내지만 이 펌웨어는 `rc` 만 받는다 → **조종 입력이 전혀 전달되지 않는다.**
- 텔레메트리가 22필드라 65필드를 기대하는 표시·기록이 대부분 빈 값이 된다 (공유 파서가 legacy 길이를 허용하므로 크래시는 나지 않는다).
- `gains` 회신이 없어 게인 읽기가 타임아웃된다.
- RC 상실 시 **자동착륙 없이 즉시 컷한다.**
- yaw 부호가 반대라 회전 방향이 뒤집힌다.

첫 비행 전 `firmware/flight/dual_imu_flix_quat_pwm/README.md` 의 벤치 부호 점검을 반드시 수행해야 한다.

7. 진단 펌웨어: firmware/diagnostics

진단 프로펠러를 제거한 벤치에서 하드웨어를 한 요소씩 확인하는 7개 스케치다. PID나 믹서를 포함하지 않으며, CI 컴파일 대상이다. docs/power_on_bench_procedure.md 의 단계별 절차와 짝을 이룬다.

7.1 모터 계열 진단 3종

motor_id_single — 단일 모터 식별 (벤치 Stage A, 권장 시작점)

함수	역할과 기제
writeMotorUs(pin, us)	비행 펌웨어의 writeMotor 와 완전히 동일한 LEDC 변환. 400Hz/14bit, duty = us×16383/2500 .
writeAllIdle()	네 핀 모두 1000μs.
applyOutputs()	setpoint를 [1000, 1250] 으로 클램프하고, 전부 아이들로 만든 뒤 선택된 하나만 setpoint로 올린다. 이 순서가 두 모터가 동시에 도는 것을 구조적으로 막는다.
selectMotor(i)	모터 선택 + setpoint를 1120으로 리셋 + 예상 회전 방향 출력.
adjust(d)	±10μs. 선택된 모터가 없으면 아이들 유지 후 안내.
stopAll()	선택 해제 + 전 모터 1000μs.
handle(c)	키 디스패치. 1~4 선택, 0/s 정지, ± 조정, r 리셋. 알 수 없는 키는 즉시 전 모터 정지(fail-safe 기본값). CR/LF는 무시한다.

왜 ESP32Servo가 아니라 LEDC인가

주석에 따르면 ESP32-S3에서 ESP32Servo는 LEDC 채널을 잘못 할당해 개별 GPIO가 개별 모터로 매핑되지 않는다(핀이 쌍으로 묶이거나 일부가 죽는다). 이 기체에서 실제로 관측된 문제이며, 그래서 모터 식별은 반드시 비행 펌웨어가 실제로 쓰는 LEDC 경로로 해야 한다.

esc_config_throttle — ESC TX 프로그래밍용 단일 채널 출력

함수	역할과 기제
<code>pulseUsToDuty(pulseUs)</code>	50Hz/14bit에서 반올림을 적용해 가능한 가장 가까운 펄스폭을 만든다 (<code>(us×MAX + PERIOD/2) / PERIOD</code>). 64비트 중간 계산으로 오버플로 방지.
<code>setPulseUs(requestedUs)</code>	<code>[1000, 2000]</code> 클램프. 클램프가 발생하면 요청값과 적용값을 함께 출력한다.
<code>trimWhitespace(text)</code>	in-place 앞뒤 공백 제거.
<code>processCommand(raw)</code>	<code>h =2000, l =1000, m =1500</code> , 정수 직접 입력, <code>? / help</code> . <code>m</code> 에는 Bidirectional 모드 경고가 붙는다.
<code>printProgrammingQuickReference()</code>	BLHeli TX 프로그래밍 5단계 절차를 출력한다. 회전 방향 메뉴에서 2번째(Reverse) 를 골라야 하며 3번째(Bidirectional) 를 고르면 안 된다 는 경고 포함 — Bidirectional은 최저 스로틀이 중립이 되어 위험하다.
<code>loop()</code>	줄 단위 명령 수집. 버퍼(32B) 초과 시 입력을 버리고 오류를 알린다. LEDC 하드웨어가 별도 갱신 없이 현재 값을 계속 출력하므로 루프가 비어 있어도 된다.

h 는 최대 2000μs를 계속 출력한다

일반 저속 모터 시험에 쓰면 안 된다. 프로펠러 제거, 전류 제한 전원, **설정할 ESC/모터 하나만 연결한** 상태에서만 사용한다.

motor_pwm_bench — 4채널 자동 램프

`setup()` / `loop()` / `writeAll(us)`

역할 — 네 모터를 동시에 1000→1300μs로 램프하는 자동 시험.

기제 — ESP32Servo 50Hz를 쓴다. `setup()` 에서 ESC 아밍을 위해 **7초 카운트다운** 후 `loop()` 가 램프업(5μs씩, 15ms 간격) → 2초 유지 → 램프다운 → 3초 정지를 무한 반복한다.

부팅 7초 뒤 네 모터가 자동으로 돈다. 시리얼 정지 명령이 없다

전원을 끄는 것 외에 멈출 방법이 없다. 위의 `motor_id_single` 과 성격이 완전히 다르므로 혼동하면 안 된다. 또한 ESP32Servo를 쓰므로 위에서 설명한 채널 묶임 문제가 발생할 수 있다.

7.2 IMU 계열 진단 4종

스케치	핀 배치	역할
<code>icm42670_single_raw</code>	SCK/MISO/MOSI/CS = 18/19/23/5	단일 IMU의 6축 raw 출력. 루프당 100ms 대기. 이 배치만 비행 펌웨어와 다르다. <code>setup()</code> 에서 <code>begin()</code> 반환값을 확인해 실패 시 메시지를 남긴다.
<code>icm42670_dual_raw</code>	12/13/11, CS 10/9	두 IMU의 raw를 나란히 출력. IMU2의 X-Z 부호 반전을 눈으로 확인 하는 것이 목적이다.

스케치	핀 배치	역할
board_v1_5_2_dual_imu	12/13/11, CS 10/9	PCB v1.5.2 브링업용. 두 IMU의 자이로 Z와 가속도 Z를 비교 출력한다.
icm42670_dual_loop_debug	12/13/11, CS 10/9 모터 4/5/6/7	구형 추정기의 루프 주기·필터 거동 진단. 아래 상세.

icm42670_dual_loop_debug 상세

`writeMotor(pin, us) / idleMotors()`

역할 — 네 ESC를 전 구간 1000 μ s로 고정. 안전을 위한 것이며 제어하지 않는다.

`static inline float compute_alpha(float ax, float ay, float az)`

역할 — 적응 상보필터 계수. 비행 펌웨어와 동일한 구조, 다른 값.

기제 — 임계는 같지만(0.1/0.3) alpha가 0.99/0.995/0.999로 구형 250Hz 루프 기준이다. 주력 펌웨어의 0.999/0.9995/0.9998과 비교하면 이 진단이 어떤 시점의 거동을 재현하는지 알 수 있다.

`static void measure_imu_bias(...) / calibrate_bias()`

역할 — 비행 펌웨어와 같은 2000샘플 bias 보정.

기제 — 단, 읽기 실패를 재시도하지 않고 `getDataFromRegisters`의 반환값을 검사하지 않는다. 진단용이므로 비행 펌웨어보다 방어가 약하다.

전역 `diag_loop_count`, `diag_max_dt_us`, `diag_sum_dt_us`, `diag_clamp_count`, `diag_last_alpha`를 50ms 텔레메트리 주기마다 리셋하며 누적한다. 이것이 이 스케치의 존재 이유다 — 루프가 실제로 몇 Hz로 도는지, dt 클램프가 몇 번 걸리는지, 어떤 alpha가 선택되는지를 정량적으로 측정한다. WiFi SSID가 `DUAL_IMU_DEBUG`로 다르고 텔레메트리가 별도 14필드이므로 반드시 `scripts/receive_dual_imu_debug.py`와 짝지어 쓴다.

이 진단으로 보정 부호를 판단하면 안 된다

PID와 믹서가 없으므로 "어느 방향으로 보정이 들어가는가"는 확인할 수 없다. 보정 방향은 주력 비행 펌웨어의 프롭 OFF 벤치 단계에서 확인해야 한다.

8. 보관 펌웨어: firmware/archive

8.1 보관 정책

보관 `firmware/archive/` 는 대체된 펌웨어를 **원형 그대로 보존**하는 역사 구역이다. 세 가지 규칙이 적용된다.

- **빌드하지 않는다.** C는 `flight/` 와 `diagnostics/` 만 컴파일한다. 따라서 이 코드들은 현재 틀체인에서 컴파일조차 안 될 수 있다.
- **수정하지 않는다.** 버그를 발견해도 고치지 않는다. 당시의 상태를 보존하는 것이 목적이기 때문이다.
- **안전하다고 가정하지 않는다.** 특히 모터를 구동하는 코드는 현행 안전 규칙을 따르지 않는다.

8.2 계열별 목록과 위험 표시

legacy_flight/ — 대체된 비행 제어 4종

스케치	센서	상태와 주의
<code>single_imu_pid_pwm</code>	ICM42670 ×1	대체된 단일 IMU PID.
<code>dual_imu_pid_pwm</code>	ICM42670 ×2	대체된 듀얼 IMU PID. 단순 평균과 시작 시 bias 보정만 있고, 당시 계획서에 있던 개별 IMU 폴백.런타임 bias 갱신은 구현되지 않았다 . 현행 텔레메트리의 앞 14필드 형식이 여기서 유래한다.
<code>single_imu_cascade_pwm</code>	ICM42670 ×1	대체된 캐스케이드 실험.
<code>single_imu_kalman_pid_pwm</code>	ICM42670 ×1	Kalman/PID 실험. 과도 기울기 분기에서 잠금 플래그만 세우고 모터 출력을 즉시 갱신하지 않는 결함 이 있다.

filter_experiments/ — 필터 실험

`icm42670_kalman_attitude` — Kalman 자세 추정 실험. 모터 제어 없음.

dshot/ — DShot600 프로토콜 실험 6종

스케치	상태와 주의
<code>single_motor_fixed_throttle</code>	고정 단일 모터 출력.
<code>single_motor_serial_control</code>	시리얼로 제어하는 단일 모터 실험.
<code>mpu6500_tilt_throttle</code>	MPU6500 기울기-스로틀 실험.
<code>mpu6500_pid_bench</code>	이름과 달리 PID 계산이 없다. 기울기로 출력을 고를 뿐이다. 게다가 센서 초기화 실패 에도 진행하며, 0 명령도 최종 출력에서 최소 5%로 바꾼다 . 실험용 벤치 예제가 아니다.

스케치	상태와 주의
<code>four_motor_full_throttle_unsafe</code>	⚠ 위험: 네 모터를 최대 출력 근처로 구동한다. 벤치 테스트로도 실행 금지. 파일명 자체가 경고다.
<code>motor_allocator_dual_core</code>	미완성 듀얼코어 모터 할당기. 시뮬레이션 입력 사용.

other_sensors/ — 센서 단독 실험 4종

`bmm350_read` (자기계 단독 읽기) · `board_v1_5_2_bmm350` (PCB v1.5.2 자기계 진단) · `gps_uart_passthrough` (raw NMEA 패스스루) · `us100_distance` (US100 초음파, 오래된 의존성).

other_mcus/ — 대체 MCU 실험

`stm32_f411_uart_rx` — STM32 F411 UART 수신 실험.

platformio_skeleton/ — 초기 PlatformIO 골격

프로젝트 초기의 PlatformIO 구조(`src/` , `lib/` , `include/`)다. 실제 센서 대신 시뮬레이션 값을 쓰고 옛 TCP 전송을 포함하므로 현재 코드로 사용하면 안 된다.

파일	내용
<code>src/main.cpp</code>	골격 진입점.
<code>src/sensor.cpp</code> / <code>lib/sensor.h</code>	시뮬레이션 센서 인터페이스.
<code>src/motor.cpp</code> / <code>lib/motor.h</code>	DShot 모터 할당기.
<code>src/connect.cpp</code> / <code>lib/connect.h</code>	옛 TCP 연결.
<code>src/pwm_RGB.cpp</code>	0바이트 빈 파일. 착수만 되고 구현되지 않았다.
<code>examples/motor_allocator_dshot/</code>	이 골격에 의존하는 예제 스케치.
<code>platformio.ini</code>	빌드 설정.

9. PC 지상국과 분석 도구: scripts/

PC에서 실행하는 16개 Python 도구다. 펌웨어와 UDP 프로토콜로 강하게 결합되어 있어, 텔레메트리 필드나 명령 형식을 바꾸면 반드시 함께 수정해야 한다. 그 결합을 한 곳에 모은 것이 `telemetry_schema.py` 다.

9.1 도구 지형도와 동시 실행 금지 규칙

분류	도구	역할
공유 모듈	<code>telemetry_schema.py</code>	65필드 프로토콜 정의와 파서. 모든 수신 도구가 임포트한다.
	<code>failsafe_telemetry.py</code>	자동착륙 phase·프로브 상태의 사람이 읽는 표현.
주력 지상국	<code>control_dualsense.py</code>	정상 조종·벤치의 유일한 대화형 지상국. RC 스트리밍·stdin 명령·상태 표시·CSV·raw 기록을 한 프로세스에서 처리.
수신·모니터	<code>receive_telemetry.py</code>	터미널 수신기 겸 CSV 로거.
	<code>monitor_telemetry.py</code>	실시간 플롯 겸 CSV 로거.
	<code>receive_dual_imu_debug.py</code>	듀얼 IMU 루프 진단 전용 14필드 수신기.
분석	<code>analyze_flight_log.py</code>	20Hz CSV 오프라인 분석과 4단 플롯.
	<code>analyze_probe_response.py</code>	자동착륙 프로브 판정·응답 분포 분석.
	<code>decode_imu_raw.py</code>	ZIMU / ZCAL 바이너리 → CSV 디코더.
	<code>magcal_fit.py</code>	캡처 CSV → 제약 타원체 hard/soft-iron 상수 피팅.
벤치 자동화	<code>bench_sign_test.py</code>	roll/pitch 보정 부호 검증.
	<code>bench_yaw_test.py</code>	yaw 보정 부호 검증.
	<code>bench_thrust_ramp.py</code>	스로틀 레벨별 모터 응답 계단 시험.
보조	<code>tune_pid.py</code>	게임패드 없이 UDP 명령·게인만 다루는 도구.
	<code>gamepad_probe.py</code>	게임패드 축·버튼 번호 실측.
	<code>test_dualsense_input.py</code>	DualSense 입력 최소 확인.
	<code>motor_serial.py</code>	시리얼로 모터 진단 스케치를 조작.
보관	<code>archive/</code> 2종	옛 GPS UDP 수신기, 옛 TCP 테스트.

두 개 이상을 동시에 실행하면 안 된다

2.3절에서 설명했듯 펌웨어는 들어온 모든 UDP 패킷의 발신자로 텔레메트리 목적지를 재지정한다. 보조 도구가 명령을 보내는 순간 텔레메트리를 가져가므로, 주력 지상국은 데이터를 잃고 로그에 구멍이 생긴다. 게임패드도 USB로 연결하고, 드론에 UDP를 보내는 프로그램은 하나만 띄운다.

9.2 telemetry_schema.py — 공유 프로토콜 파서

295행이지만 이 저장소에서 가장 중요한 Python 파일이다. 모듈 docstring 47행이 65개 필드의 의미와 함정을 전부 기술하며, 사실상 프로토콜 규격서 역할을 한다.

상수	내용
TELEMETRY_FIELDS	65개 필드명 튜플. 순서가 곧 wire 포맷이다.
TELEMETRY_FIELD_TYPES	필드명 → float / int 매핑.
GAIN_FIELDS	GAINS, 회신의 12개 게인명.
CSV_FIELDS	("Timestamp",) + TELEMETRY_FIELDS = 66열.
REQUIRED_FIELD_COUNT	10. 앞 10개는 비어 있으면 안 된다.

```
def parse_telemetry_packet(line)
```

역할 — legacy든 현행이든 모든 길이의 패킷을 고정 스키마 dict로 변환.

기제 — 이 함수의 설계가 **append-only 계약을 실제로 동작하게 만든다**. ① dict.fromkeys(TELEMETRY_FIELDS) 로 65개 키를 모두 None으로 먼저 만든다. 따라서 어떤 길이의 패킷이 와도 반환 dict의 구조는 동일하다. ② 도착한 필드 수만큼 채우고 나머지는 None으로 남긴다 → CSV에서 빈 셀이 된다. **없는 값을 "정상"이나 "고장"으로 지어내지 않는다**. ③ 알려진 65개를 넘는 미래 필드는 무시한다 → 구버전 도구가 신버전 펌웨어의 패킷을 받아도 깨지지 않는다. ④ 단, 앞 10개 필드가 비어 있으면 **예외를 던진다**. 소비자가 이 값들로 산술-포맷을 하므로, 여기가 비었다면 legacy가 아니라 **손상된 패킷**이다. ⑤ float은 isfinite, int는 정수성까지 검사한다. 이 구조 덕분에 10:14:21:22:30:31:34:35:38:40:43:55:58:59:64필드의 과거 패킷이 모두 그대로 파싱된다.

```
def is_gains_packet(line) / def parse_gains_packet(line)
```

역할 — gains 명령의 회신 식별과 파싱.

기제 — "GAINS," 접두사로 구분한다. 파싱 시 필드 수가 정확히 13개(GAINS + 12)여야 하며, 하나라도 다르면 예외를 던진다. 텔레메트리와 달리 **부분 수용을 허용하지 않는다** — 게인은 전부 알거나 전혀 모르거나 둘 중 하나여야 한다.

```
def sample_to_csv_row(timestamp, sample) / def active_fault_names(sample)
```

역할 — CSV 행 생성과 활성 고장 이름 목록.

기제 — None은 빈 문자열로 바꿔 열 수를 보존한다. active_fault_names는 6개 고장 필드 중 값이 1인 것의 축약명(RC, CRIT, IMU1, IMU2, DISAGREE, TILT)을 반환한다.

docstring이 기록한 세 가지 함정

- 텔레메트리 패킷은 원자적 스냅샷이 아니다. IMU별 12개 값은 같은 스냅샷을 공유하지만, 다른 필드는 다른 tick에서 올 수 있다. 따라서 "융합값 = IMU1·IMU2 평균"이 행 단위로 반올림 오차 내에 성립한다는 보장은 없다.
- `TgtAngle_Roll/Pitch` 에는 트림이 이미 포함되어 있다. 분석 시 `Trim_Roll` 을 다시 더하면 이중 계산이 된다.
- `MagHeading` 으로 융합 활성 여부를 판단할 수 없다. 융합이 꺼져도 마지막 값을 유지하기 때문이다. 유일한 진실은 `Mag_Enabled` (필드 59)다.

9.3 control_dualsense.py — 주력 대화형 지상국

1,140행. 세 개의 실행 흐름(메인 stdin 루프 + 텔레메트리 스레드 + 컨트롤러 스레드)이 공유 상태를 다루므로, 락 순서가 명시적으로 규정되어 있다.

허용된 락 순서: `safety_cmd_lock` → `telem_lock` (역순 금지)
금지: `telem_lock` 보유 중 `send_cmd/reliable_send` 같은 블로킹 I/O 호출

송신·안전 함수군

```
def send_cmd(cmd) / def reliable_send(cmd)
```

역할 — UDP 단발 전송 / 중요 명령의 반복 전송.

기제 — `reliable_send` 는 20ms 간격으로 5회 보낸다. UDP는 재전송이 없으므로 `start · stop` 처럼 놓치면 안 되는 명령에만 쓴다.

```
def send_trim()
```

역할 — 트림을 절대값으로 한 번만 보낸다.

기제 — docstring이 `reliable_send` 를 쓰면 안 되는 이유를 설명한다: 그것은 5회 `sleep` 하며 도는데, 이 함수는 `rcr` 을 20Hz로 보내는 `controller_thread` 에서 호출되므로 그 동안 **RC 업링크가 통째로 멈춘다**. 대신 유실 대비는 `telemetry_thread` 의 **닫힌 루프**로 처리한다(아래 참조). 트림은 증분이 아니라 절대값 명령이므로 재전송이 안전하다.

```
def disarm(reason) / def stop_streaming_only(reason)
```

역할 — 두 종류의 해제. 이 구분이 안전상 결정적이다.

기제 — `disarm` 은 `stop` 을 드론에 보내 모터를 끈다. `stop_streaming_only` 는 `stop` 을 보내지 않고 로컬 스트리밍만 멈춘다. 후자가 필요한 이유: 드론이 자동착륙 중인데 `stop` 을 보내면 **하강 중인 기체의 모터를 공중에서 꺼버린다**. 업링크만 죽는 비대칭 고장에서는 링크가 간헐적으로 열려 그 `stop` 이 실제로 도착할 수 있다. 그래서 텔레메트리 끊김·컨트롤러 분리·드론 RC 타임아웃 감지 시에는 반드시 이쪽을 쓴다.

```
def arm()
```

역할 — 시동.

기제 — 스로틀을 1100으로 맞추고 `mag <0|1>` 을 **start 보다 먼저** 보낸다. 무장 상태에서는 펌웨어가 자기계 최초 초기화를 거부하므로, 아직 disarmed인 이 시점에 보내야 부팅 후 첫 arm에서도 초기화가 통과한다. `last_arm_time` 을 기록해 이후 `Armed` 필드 대조에 쓴다.

resume 상태기계 — 4단계 확인 프로토콜

자동착륙 중 조종을 되찾는 `resume` 은 **맹목적으로 보내지 않는다**. 지상국이 텔레메트리로 여러 조건을 확인한 뒤에만 실제 명령을 발사한다.

상태	수행하는 확인
<code>idle</code>	대기.
<code>wait_rc</code>	여전히 <code>Failsafe_Phase == 1</code> 인가? 그리고 새 RC가 드론에 실제로 수락됐는가 를 <code>RC_Total_Pkts</code> 와 <code>RC_Dropped_Pkts</code> 의 증분 비교로 확인한다 (<code>total_delta > dropped_delta</code> 여야 수락). 2초 안에 확인되지 않으면 실패.
<code>send_pending</code>	토큰(<code>resume_attempt_id</code>)이 아직 유효한지 재확인한 뒤 <code>reliable_send("resume")</code> .
<code>wait_phase</code>	<code>Failsafe_Phase</code> 가 0으로 돌아왔는가? 성공하면 <code>Hover_Est</code> 를 읽어 로컬 스로틀을 드론의 호버 추정치와 동기화 한다. 1초 안에 확인되지 않으면 실패.

```
def _forward_counter_delta(current, baseline)
```

역할 — uint32 카운터의 **정방향 증가만** 인정.

기제 — `(current - baseline) & 0xFFFFFFFF` 를 구하고, 결과가 `0x80000000` 이상이면 `None` 을 반환한다. 즉 wrap을 포함한 작은 전진은 인정하되 **rollback은 거부**한다. 드론이 재부팅해 카운터가 리셋되면 이것이 감지되어 `resume`이 취소된다 — 재부팅한 드론에 `resume`을 보내는 것은 위험하기 때문이다.

```
def _invalidate_resume_attempt() / def cancel_resume_attempt() / def check_resume_timeout(now)
```

역할 — 세대 토큰 기반 취소와 벽시계 데드라인 집행.

기제 — `resume_attempt_id` 를 증가시켜 진행 중인 시도를 **세대 단위로 무효화**한다. 킬이나 링크 재상실 뒤 늦게 도착한 텔레메트리가 이미 취소된 `resume`을 발사하는 것을 막는다. `check_resume_timeout` 은 **텔레메트리 수신 여부와 무관하게** 데드라인을 집행한다 — 패킷이 아예 안 오면 상태기계는 영원히 멈춰 있을 수 있기 때문이다.

telemetry_thread — 수신·기록·감시

```
def telemetry_thread()
```

역할 — 패킷 수신, CSV 기록, 상태 표시, 그리고 **8종의 자동 안전 반응**.

기제 — 매 반복의 처리 순서는 다음과 같다. ① **주기적 connect 전송** — 1초마다 보낸다. 시동 전에도 텔레메트리를 받으려면 드론에 목적지를 등록해야 하는데, 펌웨어는 **어떤 패킷이든** 발신자를 목적지로 삼으므로 의미 없는 문자열이면 충분하다 (펌웨어에서 `connect` 는 어떤 명령에도 매치되지 않고 게인 파서에서 조용히 버려진다). ② **텔레메트리 타임아웃 감시** — 1.5초 무수신이면 스트리밍 중일 때 `stop_streaming_only` 로 RC를 끊어 **드론의 자동착륙에 맡긴다**. ③ **패킷 종류 분기** — ZIMU / ZCAL 은 `_append_raw_datagram` 으로 `.bin` 에 추가, GAINS, 는 게인 출력, 나머지는 텔레메트리로 파싱. 디코드·파싱 실패는 조용히 버린다. ④ **트림 닫힌 루프** — 조종자가 아직 트림을 건드리지 않았으면 **드론이 권위**여서 드론 값을 받아들인다. 조종자가 한 번이라도 바꾼 뒤에는 **지상국이 권위**여서, 드론 값이 0.01° 넘게 다르면 200ms 간격으로 재전송한다. 전송은 반드시 락 밖에서 한다. ⑤ **CSV 기록** — 20패킷마다 `flush` . 비정상 종료에도 최대 1초치만 잃는다. ⑥ **상태·자동착륙 출력** — 1초마다 요약. 자동착륙은 `DESCENDING` 이면 120ms 프로브 딥을 놓치지 않도록 **매 패킷** 출력하고, 갱신 정보가 없는 종료 phase(2/3/4)는 전이 시점에만 한 번 출력한다. ⑦ **진단 라인** — 무장 중 4패킷마다(약 5Hz) 모터 μs 4개, **CW-CCW 대각쌍 차이(yaw축)**, 자세, 각도 오차, 자이로, IMU 불일치, 트림, AGL, yaw hold, 목표 rate, 호버 추정을 한 줄로 출력한다. ⑧ **8종 자동 반응** — 아래 표.

감지 조건	자동 반응
텔레메트리 1.5초 무수신	<code>stop_streaming_only</code> (드론 자동착륙에 위임)
<code>Armed == 0</code> 인데 로컬은 무장 (1.5초 grace 후)	<code>disarm</code> . 시동이 거부됐음을 감지한다 — 펌웨어의 "START REFUSED"는 시리얼에만 나오므로 지상국은 이 필드로만 알 수 있다.
<code>Fault_RC</code> 상승 엣지	<code>stop_streaming_only</code>
<code>Fault_Critical</code> 상승 엣지	<code>disarm</code> + 활성 고장명 출력
컨트롤러 분리 (controller_thread)	<code>stop_streaming_only</code> + 재연결 대기 루프
패킷 드롭률 > 10%	간섭 의심 경고
$ \text{기울기} > 30^\circ$ (무장 중)	1초에 한 번 경고
resume 데드라인 초과	시도 취소 + 사유 출력

고장 플래그는 펌웨어에서 래치되므로 상승 엣지에서만 알린다. 매 패킷 출력하면 터미널이 잠긴다.

controller_thread — 20Hz 입력 처리

```
def controller_thread()
```

역할 — 게임패드를 20Hz로 읽어 `rcr` 을 보내고 스로틀·트림을 관리.

기제 — **정밀 타이밍**: `time.sleep(loop_dt)` 가 아니라 `next_loop += loop_dt` 방식의 `monotonic` 기준을 쓴다. `sleep` 오차가 누적되지 않아 실제로 20Hz가 유지된다. **스로틀을 트리거 전용으로 만든 이유**: 오른쪽 스틱 매핑을 제거했다. 스틱은 중립 위치가 드리프트하지만 **트리거는 해제 시 확실히 안 눌린 상태**이기 때문이다. R2/L2는 누르는 동안 $\pm 200\mu\text{s/s}$ 로 연속 램프, R1/L1은 $\pm 1\mu\text{s}$ 정밀 조정. **RC 명령**: 매 루프 `rc_seq` 를 증가시키고 `rcr <seq> <roll> <pitch> <yaw_rate>` 를 보낸다. 스틱은 데드존(roll/pitch 0.05, yaw 0.12)을 거쳐 각각 $\pm 15^\circ$, $\pm 90\text{dps}$ 로 매핑된다. **컨트롤러 분리 처리**: `get_count() == 0` 이면 `stop_streaming_only` 로 RC를 끊고 재연결을 기다린다. 재연결 시 타이밍 기준을 재설정한다.

입력	기능
X (b0)	시동/해제 토글 (엣지 감지)
$\triangle$ (b3)	자동착륙 중 명시적 resume
R2/L2 (a5/a2)	스로틀 연속 $\uparrow/\downarrow$ ($200\mu\text{s/s}$)
R1/L1 (b5/b4)	스로틀 $\pm 1\mu\text{s}$ 정밀
L스틱	Roll/Pitch ($\pm 15^\circ$)
R스틱 $\leftrightarrow$	Yaw 각속도 ($\pm 90\text{dps}$)
DPAD	트림 $\pm 0.2^\circ$ (최대 $\pm 10^\circ$)

입력	기능
PS (b12)	트림 리셋

```
def handle_stdin_command(msg)
```

역할 — stdin 명령을 안전 처리 후 UDP로 중계.

기제 — `resume / start / stop` 은 로컬 상태기계를 거치고, `raw / mag` 는 `on / off` 를 0/1로 변환하며, `th` 는 로컬 스로틀 변수까지 동기화한다. **그 외 모든 문자열은 그대로 UDP로 전달**되므로 PID 계인 명령이 별도 처리 없이 동작한다.

```
def shutdown_workers_and_close_log(workers)
```

역할 — 종료 시 스레드 정리, 로그 닫기, `raw` → CSV 자동 변환.

기제 — 각 스레드를 2초 타임아웃으로 join하고, 살아남으면 경고한다. CSV를 flush-close한 뒤 `.bin` 이 있으면 `decode_file()` 을 호출해 CSV로 변환하고 통계(배치 수, 샘플 수, 무선 유실, 생산자 드롭, 평균 샘플레이트, dt 최소/최대)를 출력한다. 변환 실패해도 **`.bin` 은 반드시 남긴다.**

표시 헬퍼 함수군

함수	역할
<code>_format_optional_float/int , _format_binary_flag</code>	<code>None</code> ·비유한값을 <code>-</code> 로 표시. legacy 패킷에서 빈 값을 안전하게 출력.
<code>_format_agl(sample)</code>	거리계 값 표시. <code>Range_Quality == -1</code> 이면 값을 신뢰하지 않는다는 규칙을 여기서 구현한다.
<code>_format_drone_trim(sample)</code>	드론이 보고한 트림 표시(지상국 값과 대조용).
<code>_format_angle_error(target, measured)</code>	목표-측정 각도 오차.
<code>_format_gyro_disagreement(sample)</code>	IMU1·IMU2 자이로 차이의 최댓값.
<code>format_rc_timeout_fault(sample)</code>	RC 타임아웃 시 호버 추정·스로틀 상태로 어느 분기를 탈지 설명.
<code>format_autoland_telemetry(sample)</code>	하강 중 phase·프로브 상태·응답 한 줄.
<code>format_status_telemetry(sample)</code>	1초 주기 링크·상태 요약.
<code>_format_raw_status() , _append_raw_datagram(data)</code>	raw 캡처 상태 표시와 <code>.bin</code> 누적 기록.

9.4 failsafe_telemetry.py — 자동착륙 상태 표시

함수/상수	역할과 기제
<code>FAILSAFE_PHASE_NAMES</code> <code>FAILSAFE_PROBE_STATE_NAMES</code>	펌웨어 열거값 → 사람이 읽는 이름. 펌웨어 헤더의 wire 계약과 1:1 대응해야 한다.

함수/상수	역할과 기제
<code>phase_number(value)</code>	문자열/실수/None을 안전하게 정수 phase로. CSV에서 읽은 값이 어떤 형태든 처리한다.
<code>format_failsafe_phase</code> <code>format_failsafe_probe_state</code>	알 수 없는 값도 <code>UNKNOWN(n)</code> 형태로 표시해 새 펌웨어 값을 조용히 삼키지 않는다.
<code>_format_trim /</code> <code>_format_hover_status /</code> <code>_format_probe_status</code>	각각 트림, 호버 추정(값+유효 플래그), 프로브(상태+무반응 횟수+최근 응답)를 한 조각으로.
<code>format_monitor_status(sample)</code>	위 조각들을 합친 요약 문자열.
<code>render_monitor_title(axis,</code> <code>system_summary, sample)</code>	matplotlib 축 제목에 상태를 실시간 반영. <code>monitor_telemetry.py</code> 가 사용한다.

9.5 decode_imu_raw.py — 바이너리 디코더

펌웨어가 보낸 `.bin` 을 CSV로 변환한다. 374행.

구조	내용
<code>ZIMU_HEADER</code>	<code>struct.Struct("<4sBBHIII")</code> = 20B. 펌웨어의 <code>static_assert</code> 와 일치.
<code>ZIMU_SAMPLE_BY_VERSION</code>	v1(26B) / v2(27B) 두 레이아웃을 모두 지원.
<code>ZCAL_PACKET</code>	<code>"<4sB3x13f"</code> = 60B.
<code>Calibration ,</code> <code>DecodedSample ,</code> <code>DecodeSummary</code>	dataclass 3종. 보정 상수, 샘플 하나, 전체 통계.

```
def _parse_capture(data) -> (samples, summary)
```

역할 — 바이트 스트림을 샘플 목록으로. **손상 복원이 핵심 책임.**

기제 — UDP 캡처는 패킷이 잘리거나 섞일 수 있으므로 방어가 여러 겹이다. ① 매직이 아니면 `_next_magic` 으로 다음 ZIMU / ZCAL 경계까지 건너뛰고 `unknown_magic_records` 를 센다. ② 레코드 끝이 데이터 길이를 넘거나, **레코드 내부에 다음 매직이 나타나면** 잘린 것으로 보고 `truncated_records` 를 센다. ③ **버전을 먼저 읽는다** — v1과 v2의 샘플 크기가 다르므로 버전을 모르면 레코드 길이를 계산할 수 없다. 모르는 버전은 건너뛴다. ④ `batch_seq` 의 uint32 전진 차이로 **무선 유실**을 집계하고, 헤더의 `dropped` 로 **펌웨어 큐 넘침**을 따로 기록한다. 두 손실 원인을 구분하는 것이 이 디코더의 중요한 기능이다. ⑤ 시각 복원: `base_t_us` 에서 시작해 두 번째 샘플부터 `dt_us` 를 누적하며 `& 0xFFFFFFFF` 로 wrap을 처리한다. ⑥ **ZCAL 소급 적용**: 첫 ZCAL 을 만나기 전의 샘플들에 `dataclasses.replace` 로 첫 보정값을 채워 넣는다. 캡처를 중간부터 시작해도 물리량을 복원할 수 있다. ⑦ 평균 샘플레이트는 **첫 샘플을 제외하고** 계산한다 — 첫 샘플의 `dt` 는 이전 배치와의 간격이라 의미가 다르기 때문이다.

```
def _calibration_from_packet(values) / def _physical_values(...) / def _write_csv(...) / def _print_summary(...) / def decode_file(...) / def main(argv)
```

역할 — ZCAL 언패킹, raw→물리량 변환, CSV 출력, 통계 출력, 파일 단위 API, CLI.

기제 — `_physical_values` 는 펌웨어와 **동일한 순서**로 IMU2 부호 → bias 차감 → 스케일을 적용한다. 이 순서가 어긋나면 오프라인 재현이 실기와 달라진다. `decode_file` 은 `control_dualsense.py` 가 종료 시 직접 호출하는 진입점이다.

9.6 magcal_fit.py — 자기계 타원체 피팅

527행. **Li & Griffiths 제약 타원체 피팅**으로 hard/soft-iron 상수를 구한다. 이 저장소에서 수학적으로 가장 정교한 코드다.

원리

이상적인 자기계를 모든 방향으로 돌리면 측정점이 **원점 중심의 구**를 그린다. 실제로는 기체의 자성체가 오프셋(hard-iron)을 만들고 주변 강자성체가 왜곡 (soft-iron)을 일으켜 **중심이 이동한 타원체**가 된다. 측정점에 타원체를 피팅해 중심(hard-iron)과 구로 되돌리는 변환 행렬(soft-iron)을 구하는 것이 목표다.

```
def _coverage(values) -> (eigenvalues, ratio)
```

역할 — 샘플이 **충분히 다양한 방향**을 덮는지 검증.

기제 — 중심화한 샘플을 단위벡터로 정규화하고 그 공분산의 고윳값을 구한다. 최대/최소 비율 이 100 이상이면 `CoverageError` 로 **거부**한다. 한 평면에서만 돌린 데이터로는 3축 타원체가 결정되지 않기 때문이다. 이 검사가 없으면 수학적으로는 답이 나오지만 물리적으로 무의미한 결과를 얻는다.

```
def _design_matrix(values) / def _constraint_matrix()
```

역할 — 일반 2차곡면 $Ax^2+By^2+Cz^2+2Dxy+2Exz+2Fyz+2Gx+2Hy+2Iz+J=0$ 의 설계행렬과 Li & Griffiths 제약행렬.

기제 — 제약은 $4J - I^2 = 1$ 이다. 이 제약이 **해가 타원체가 되도록 강제한다** — 제약 없이 최소제곱을 풀면 쌍곡면이나 원뿔로 수렴할 수 있다.

```
def _positive_generalized_eigenvector(scatter, constraint)
```

역할 — 일반화 고유값 문제 $Sv = \lambda Cv$ 를 푼다.

기제 — docstring이 문제를 정확히 설명한다: 제약행렬은 양의 방향 하나와 음의 방향 다섯을 가지므로 **양정치가 아니어서** 일반적인 `scipy.linalg.eigh` API에 metric으로 넘길 수 없다. 대신 **scatter 행렬을 Cholesky 분해해 화이트닝**하고 $L^{-1} C L^{-T} y = (1/\lambda) y$ 라는 **대칭** 문제로 바꿔 푼다. 양의 고윳값이 정확히 하나가 아니면 `FitError` 로 거부하고, 얻은 벡터를 $v^T C v = 1$ 로 정규화한다.

```
def _fit_unit_ellipsoid(samples) -> _UnitEllipsoid
```

역할 — 실제 피팅 수행.

기제 — 6단계. ① **수치 정규화** — 평균을 빼고 RMS 반경으로 나눈다. μT 단위의 원값을 그대로 쓰면 설계행렬의 조건수가 나빠진다. ② **블록 소거** — 10×10 산포행렬을 2차항(6)과 1차항+상수(4)로 나눠 $S11 - S12 \cdot S22^{-1} \cdot S12^T$ 로 축소한다. ③ 제약 고유벡터를 구해 `beta` 10개를 복원한다. ④ **부호 정규화** — `trace < 0` 이면 전체 부호를 뒤집는다. ⑤ **양정치 검증** — 고윳값이 0 이하면 타원체가 아니므로 거부. ⑥ **화이트닝 행렬** — 고유분해 후 $v \cdot v \Lambda \cdot v^T$ 로 대칭 제곱근을 만들고 정규화 스케일을 되돌린다. 각 단계마다 `isfinite` 검사가 붙어 있다.

```
def fit_calibration(samples) -> CalibrationResult
```

역할 — 피팅 + 이상치 제거 + 4단계 품질 검증.

기제 — 반복 MAD 클리핑을 3회 수행한다. 매회 피팅 후 반경 잔차의 **중앙값 절대편차**를 구하고 $\sigma = 1.4826 \cdot \text{MAD}$ (정규 분포 환산)를 계산해 $|\text{잔차}| > 3\sigma$ 인 점을 버린다. 남은 inlier가 전체의 70% 미만이면 거부한다. **평균이 아니라 중앙값 기반**을 쓰는 이유는 이상치 자체가 평균을 오염시키기 때문이다. 최종 검증: 보정 후 $|B|$ 의 산포가 5%를 넘으면 "샘플이 하나의 타원체에 맞지 않는다"며 거부한다. 반환 결과에는 보정 전후 산포, 반경 잔차 95퍼센타일, 커버리지 비율, 목표 반경이 함께 담긴다.

나머지 함수	역할
<code>apply_calibration(samples, result)</code>	<code>soft_iron @ (raw - hard_iron)</code> 적용. 1개~N개 모두 지원.
<code>load_calibration_csv(path, use_all)</code>	비행 CSV에서 <code>Mag_X/Y/Z</code> 를 읽는다. 기본은 <code>Mag_Cal_Active == 1</code> 인 행만 선택한다 — 그때만 raw 도메인이기 때문이다.
<code>format_firmware_constants(result)</code>	결과를 펌웨어에 붙여넣을 수 있는 C 배열 리터럴 로 출력. 수동 전사 오류를 없앤다.
<code>recalibrate_mag_comp(...)</code>	soft-iron 좌표계 변경에 맞춰 기존 throttle 간섭 계수를 $w @$ 기존값으로 환산 한다 (5.3절 참조).
<code>format_report(result)</code>	샘플 수, inlier 수, 커버리지, 산포, 잔차, 반경의 진단 보고.
<code>_parser()</code> / <code>main(argv)</code>	CLI.

9.7 분석 도구 2종

analyze_flight_log.py — 20Hz CSV 오프라인 분석

함수	역할과 기제
<code>select_log_file()</code>	인자가 없으면 <code>logs/</code> 에서 최신 파일을 고르거나 목록에서 선택하게 한다.
<code>asserted_samples(column, invert)</code>	플래그 열이 1인(또는 0인) 샘플 수 집계.
<code>latest_number(column)</code>	열의 마지막 유효 숫자값.
<code>timestamp_seconds(value)</code>	<code>HH:MM:SS.mmm</code> → 초. 파싱 실패는 <code>None</code> .
<code>collect_failsafe_analysis(frame)</code>	자동착륙 에피소드 추출. phase 전이 목록, DESCENDING 구간의 시작·끝·소요시간·종료 phase를 뽑는다. 자정 넘김 보정이 들어 있다: 시각이 이전보다 12시간 이상 작아지면 하루(86400초)를 더한다. 로그가 자정을 넘기면 소요시간이 음수가 되는 문제를 막는다. 로그 끝까지 DESCENDING이면 "종료 phase 없음"으로 따로 보고한다.

모듈 레벨에서 4단 플롯(`matplotlib`)을 그린다. `--no-plot`으로 생략할 수 있다.

analyze_probe_response.py — 프로브 응답 분포 분석

함수	역할과 기제
<code>read_firmware_float_constant(name, path)</code>	펌웨어 소스를 직접 파싱해 상수를 읽는다. 분석 스크립트에 임계값을 하드 코딩하면 펌웨어와 어긋나므로, 소스를 단일 진실로 삼는 구조다.
<code>ProbeEvent</code> / <code>EventSummary</code> / <code>ProbeExtraction</code>	dataclass 3종.
<code>_optional_float/_optional_int(raw, field, row)</code>	행 번호를 포함한 오류 메시지로 파싱.
<code>_event_from_row(...)</code>	CSV 한 행에서 프로브 이벤트 추출.
<code>extract_probe_events_with_stats(csv_path, label)</code>	EVALUATE 전이를 찾아 이벤트화하고 제외 사유별 통계를 함께 반환.
<code>summarize_events(events)</code>	개수·평균·분위수 요약.
<code>_hover_groups(events)</code> / <code>check_hover_consistency(events)</code>	호버 추정치가 <code>HOVER_TOLERANCE_US</code> (1 μ s) 내로 일관된지 확인. 다른 호버 조건의 데이터를 섞어 비교하는 오류를 막는다.
<code>print_label_summary(...)</code> / <code>print_distribution_judgement(...)</code>	라벨(지면/공중)별 요약과 분리 가능성 판정. 최소 샘플 20개를 요구한다.
<code>plot_distributions(events, threshold)</code>	임계선을 겹친 히스토그램.
<code>_labels_for_paths(paths, labels)</code> / <code>build_argument_parser()</code> / <code>main(argv)</code>	라벨 매칭과 CLI.

5.1절에서 설명한 "프로브를 판정에서 뺀다"는 결론이 바로 이 도구의 출력으로 내려졌다.

9.8 수신·모니터 도구 3종

도구	역할과 기제
<code>receive_telemetry.py</code> (102행)	가장 단순한 수신기. UDP 4210에 바인드해 <code>parse_telemetry_packet</code> 으로 파싱하고 터미널 출력 + <code>logs/</code> 에 CSV 기록. 조종 기능이 없다.
<code>monitor_telemetry.py</code> (241행)	실시간 플롯. <code>MAX_LEN=200</code> (20Hz 기준 약 10초) 롤링 윈도우를 <code>collections.deque</code> 로 유지하고 <code>FuncAnimation</code> 의 <code>update_plot</code> 이 갱신한다. <code>optional_number</code> 로 <code>None</code> 을 안전 처리하고, 제목에 <code>render_monitor_title</code> 의 자동착륙 상태를 표시한다. CSV도 함께 기록한다.
<code>receive_dual_imu_debug.py</code> (136행)	<code>icm42670_dual_loop_debug</code> 전용 14핀 수신기. <code>WINDOW=40</code> (약 2초) 롤링 통계로 루프 주기·dt 클램프·alpha 분포를 표시한다. SSID가 <code>DUAL_IMU_DEBUG</code> 로 다르고 형식도 다르므로 다른 수신기와 호환되지 않는다.

9.9 벤치 자동화 3종

`bench_sign_test.py` (349행), `bench_yaw_test.py` (337행), `bench_thrust_ramp.py` (311행)는 거의 같은 골격을 공유한다.

공통 함수	역할과 기제
<code>nmcli_up(ssid)</code>	Linux NetworkManager로 WiFi를 자동 전환 한다. 드론 AP와 집 네트워크를 오가며, 측정이 끝나면 원래 SSID로 복원한다.
<code>wait_reachable(timeout)</code>	드론이 응답할 때까지 대기(최대 25초).
<code>sender()</code> / <code>receiver()</code>	RC 스트리밍 스레드와 텔레메트리 수신 스레드.
<code>collect_window(seconds)</code>	지정 시간 동안 샘플을 모아 반환.
<code>mean(vals)</code> / <code>std(v)</code> / <code>motor_means(samples)</code>	통계 계산.
<code>cleanup_and_restore()</code> / <code>sigint(*_)</code>	어떤 경로로 끝나도 <code>stop</code> 을 보내고 네트워크를 복원 한다. Ctrl+C 핸들러도 같은 경로를 탄다.
<code>save_results(verdict_lines)</code>	판정 결과를 파일로 저장.
<code>main()</code>	PHASES 목록을 따라 음성 안내 → 카운트다운 → 측정 → 판정.

도구	고유 판정 로직
<code>bench_sign_test.py</code>	기체를 손으로 기울였을 때 내려간 쪽 모터가 올라간 쪽보다 최소 4μs 높아야 PASS . 기울기가 6° 미만이면 "기울임 부족"으로 무효 처리한다. <code>th 1150</code> 으로 낮은 스로틀을 쓴다.
<code>bench_yaw_test.py</code>	기체를 비틀었을 때 CW 프로펠러 쌍(M1,M2)과 CCW 쌍(M3,M4)의 차이가 복원 방향으로 나오는지 확인. 10° 미만은 무효.
<code>bench_thrust_ramp.py</code>	1050μs부터 CAP(기본 1300, 하드 상한 1500)까지 50μs 계단으로 올리며 각 레벨을 2.5초 유지하고 모터 응답의 평균·표준편차를 기록한다. 매 레벨 고장 플래그를 검사한다.

세 도구 모두 시작 시 `trim 0 0` 을 보내 드론의 트림을 지운다 — 트림이 남아 있으면 모터 μs 비교가 오염되기 때문이다.

9.10 보조 도구와 보관 도구

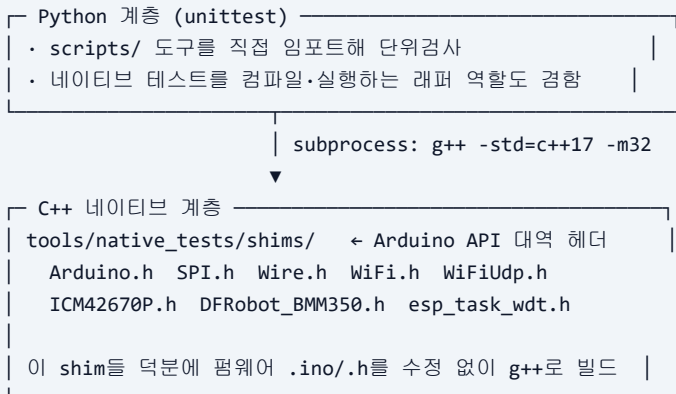
도구	역할과 기제
<code>tune_pid.py</code> (87행)	게임패드 없이 UDP 명령·게인만 다룬다. <code>receive_thread()</code> 가 텔레메트리를 받아 1초마다 상태를 출력하고, 메인은 stdin을 그대로 UDP로 보낸다. 주력 지상국과 동시 실행 금지 .
<code>gamepad_probe.py</code> (97행)	게임패드의 실제 축·버튼 번호를 실측한다. <code>snapshot(joy)</code> 로 현재 상태를 찍고 <code>print_full(joy)</code> 로 전체 출력. <code>AXIS_CHANGE_THRESHOLD=0.15</code> 로 스틱 노이즈를 무시한다. <code>control_dualsense.py</code> 의 매핑 주석 (R1=b5, L1=b4, R2=a5, L2=a2)이 이 도구의 결과다.

도구	역할과 기제
<code>test_dualsense_input.py</code> (57행)	DualSense 연결·입력 최소 확인.
<code>motor_serial.py</code> (76행)	시리얼(<code>/dev/ttyACM0</code> , 115200)로 모터 진단 스케치를 조작한다. <code>reader()</code> 스레드가 응답을 출력하고 메인인 <code>stdin</code> 을 전달한다. UDP가 아니라 시리얼 이므로 위 동시 실행 금지 규칙과 무관하다.
<code>archive/receive_gps_udp_legacy.py</code>	보관 옛 GPS UDP 수신기.
<code>archive/test_tcp_legacy.py</code>	보관 옛 TCP 테스트. <code>send_command()</code> 로 <code>192.168.0.101:8888</code> 에 접속하던 시절의 코드다. 현재 구조는 UDP SoftAP다.

10. 검증 도구: tools/

기체 없이 PC에서 돌리는 검증 계층이다. Python 테스트 14개(175개 테스트 케이스)와 C++ 네이티브 테스트 6개로 구성된다. **이 저장소의 가장 특징적인 부분**으로, 펌웨어 헤더와 `.ino` 본체를 호스트에서 직접 컴파일해 제어 수학을 단위검사하고 페루프 시뮬레이션까지 수행한다.

10.1 2계층 검증 구조



왜 -m32 로 빌드하는가

ESP32-S3와 같은 정수·포인터 폭을 쓰기 위해서다. 64비트로 빌드하면 `long` 이 8바이트가 되어 `uint32 wrap` 동작, 구조체 크기 `static_assert`, 시퀀스 번호 산술이 실기와 달라진다. `test_sil_attitude.py:39` 의 틀체인 탐지가 `static_assert(sizeof(long) == 4)` 로 이를 직접 확인한다.

shims — Arduino API 대역 구현 7종

shim	제공하는 것
Arduino.h	PI, constrain, min/max, millis() / micros() (테스트가 제어하는 가짜 시계), Serial, ledcWrite, FreeRTOS 흉내 (vTaskDelayUntil 등). 표준 라이브러리 위에 구현한다.
WiFiUdp.h	wifi_udp_fake 네임스페이스에 telemetry_output 문자열을 두어, 펌웨어가 보낸 텔레메트리를 테스트가 그대로 읽는다. incoming_packet 으로 명령을 주입할 수도 있다.
ICM42670P.h	센서 이벤트 구조체와 가짜 드라이버. 테스트가 raw 값을 주입한다.
DFRobot_BMM350.h	자기계 가짜 드라이버.
WiFi.h, SPI.h, Wire.h, esp_task_wdt.h	no-op 또는 최소 동작 스텝.

10.2 네이티브 테스트 6종

모두 `runCase(name, body)` 패턴으로 케이스를 등록하고 `main()` 이 실행한다. 실패 시 0이 아닌 종료 코드를 반환해 Python 래퍼가 감지한다.

test_control_math.cpp (2,397행, 40+ 케이스)

주력 `.ino` 본체를 직접 컴파일해 믹서·RC 파서·위치독을 검사한다. 주요 검사 항목:

- **믹서** — 영 명령은 네 모터가 같다 / 순수 roll-pitch-yaw가 FL/RR/FR/RL 부호를 따른다 / 극단 명령도 모터 한계 안에 있다 / 맞는 명령은 토크 차이를 **정확히** 보존한다 / 포화 시 **하나의 공통 스케일**만 적용된다 / **collective 이동이 자세 스케일링보다 먼저** 일어난다 / `minMotor > maxMotor` 인 비정상 입력도 안전한 경계로 수렴한다.
- **writeMotor** — 1000 미만·2000 초과 값이 **변환 전에** 제한된다.
- **RC 위치독** — 미래 타임스탬프(약간·크게), 정확한 경계값, 1ms 초과, `millis()` wrap 전후 — 7개 경계 케이스.
- **RC/rcr 파서** — 중복·역행 시퀀스 폐기 / 시퀀스 간격의 드롭 집계 / **uint32 wrap이 전진으로 인정됨** / wrap 직후 중복 0 처리 / 2:3필드 벤치 형식 수용 / 5번째 인자 거부 / 부호 붙은 시퀀스 거부 / 범위 초과 거부 / **하강 중에는 회계와 위치독만 갱신하고 목표각 보존.**
- **기타** — 1000 tick이 `millis` 를 약 1000ms 전진시킨다 / 트림 절대값 저장 / `start` 가 시퀀스 유효성을 리셋한다 / margin을 넘은 자세 차동이 전달률 미달로 프로브를 폐기한다.

test_sil_attitude.cpp (2,415행, 28 케이스) — 페루프 SIL

가장 정교한 테스트다. **실제 캐스케이드 스케치를 호스트 물리 플랜트와 페루프로 연결해** 돌린다. 시뮬레이션에는 질량, 추력, 지면 효과, 접지 충격, 가속도 노이즈, 돌풍이 모델링되어 있다.

계열	검사 내용
기반	shim tick 혹 순서·한계·리셋 / 비유한 궤적은 정착할 수 없음(헬퍼 자체 검증)
캘리브레이션	IMU별 스케일이 주입된 1.007g를 보정한다 / 비현실적 가속도 크기에서는 단위 스케일 유지
텔레메트리	목표각·자기계·3901-L0X-magcal이 필드 55 뒤에 append 되는지
수직	1g 비력이 legacy IMU raw 바이트를 보존 / 1340μs에서 호버 / 0.4m/s 접촉이 얇은 펄스를 만들 / 무른 표면은 깊고 느리게 눌러 하드 플로어와 서명이 다름 / RC 단절이 실제 펌웨어 failsafe에 진입 / 지면 효과와 수직력이 비력을 스케일
자세	+30° roll-pitch 규정 입력이 150ms 안에 추종 / S1 roll-pitch 자세 유지 수렴 / S2 정착 대칭성 / S3 안티와인드업 클램프와 포화 후 복구 / S6 hold가 아닌 구간에서도 yaw 적분기 누적
자동착륙 회귀	R1 질량 1.3배·노이즈 0.08g 위상 스위치에서 공중 착륙 없음 / V1 노이즈 OFF/ON 정상 하강이 접촉 후 착륙, sd 0.02/0.04/0.06g 모두 착륙 / V2 호버 추정이 상승 중 링크 상실을 하강시킴, 호버 없으면 즉시 컷 / V3 지면 효과 계수 0.15/0.25/0.35에서 접촉 전 착륙 없음 / V4 돌풍 후 정상 하강은 접촉 후에만 착륙 / V5 지면 프로브 1회 후 바운스는 공중에서 확정하지 않음 / V6 늦은 resume은 하강 속도와 손실을 키움
진동	설정된 표준편차가 펌웨어 5Hz LPF를 통과해 살아남음

나머지 네이티브 테스트 4종

파일	주요 검사
test_failsafe_land.cpp (372행, 23 케이스)	호버 유효 시간 누적(부적합 구간 제외) / 3초 LPF 추적 / 하강 스로틀 계산과 1000 하한 / 호버 비례 딥의 반올림과 런타임 상하한 공개 / min_descend 전 프로브 미시작 / 딥 120ms 지속과 첫 30ms 과도 제외 / 연속 무반응 확정 / 공중 반응이 카운트를 지움 / 전달되지 않은 프로브는 카운트를 올리지도 내리지도 않음 / 막힌 프로브만 반복되면 착지 미확정 / 판정은 절대 1g가 아니라 차분 / 딥이 1000 μ s 아래면 UNAVAILABLE 유지 / failsafeStep 우선순위
test_mag_yaw_fusion.cpp (616행, 9 케이스)	단위 soft-iron이 legacy hard-iron 변환을 보존 / 펌웨어에 박힌 상수가 사용 가능한 보장인지 / 비대각 soft-iron이 행 단위로 적용 / magcal wire 플래그와 XYZ 도메인의 일치 / 무샘플 stop이 보장값 보존 / 회전 heading에서 수렴 / 자이로 bias 드리프트 제거 / 비영 roll-pitch에서 틸트 보상 / $\pm 180^\circ$ 경계에서 짧은 쪽 보장
test_yaw_command.cpp (172행, 13 케이스)	스틱 편향 중 rate 모드와 setpoint 슬레이빙 / 회전 중에는 미잠금 / 중립+정착에서 잠금 / 잠긴 뒤 복원 명령 / 큰 heading 드리프트로 진입해도 슬램 없음 / 오버라이드 강제 / 하드 실링 클램프 / ± 180 경계 / 잠금 latch가 각속도 왕복에도 setpoint 유지
test_msp_sensor.cpp (214행, 12 케이스)	거리계·광류 프레임 해석 / 범위 밖 음수 거리를 부호 그대로 보존 / CRC 오류 거부와 카운트 / 모르는 function 건너뛰기 / 아는 function이라도 길이가 다르면 unknown / 버퍼 초과 페이로드는 소비만 / 쓰레기 뒤 재동기화 / 헤더 도중 '\$' 를 새 프레임 시작으로 수용 / 여러 read에 걸친 프레임 재조립 / 방향 바이트 3중 / CRC가 방향 바이트를 제외

10.3 Python 테스트 13종

파일	케이스	대상
test_control_dualsense.py	98	주력 지상국. 10개 테스트 클래스로 resume 상태기계, 트림 닫힌 루프, 안전 반응, 표시 포맷, 락 순서를 검사한다. 단일 파일 최대 테스트 수.
test_telemetry_schema.py	30	파서. legacy 길이 수용, 필수 필드 검증, 타입 검사, GAINS 파싱.
test_analyze_probe_response.py	12	프로브 분석기.
test_failsafe_telemetry.py	11	phase·프로브 상태 포맷.
test_magcal_fit.py	7	타원체 피팅과 거부 조건.
test_decode_imu_raw.py	6	바이너리 디코더(잘림·버전·유실 처리).
test_repo_layout.py	4	레이아웃 검사기 자체.
test_native_control_math.py	2	네이티브 control math 래퍼 + 컴파일 가드.
test_sil_attitude.py	1	SIL 래퍼(180초 타임아웃).
test_failsafe_land.py · test_mag_yaw_fusion.py test_msp_sensor.py · test_yaw_command.py	각 1	각 네이티브 테스트의 컴파일·실행 래퍼.

```
class SilAttitudeTest.test_sil_attitude()
```

역할 — 네이티브 SIL의 컴파일·실행 래퍼. 래퍼 패턴의 대표 예.

기제 — 4단계로 진행한다. ① `g++` 가 없으면 `skipTest`. ② `-m32` **툴체인 프로브** —

`static_assert(sizeof(long)==4)` 가 들어간 최소 프로그램을 컴파일해 본다. 실패하면 `skip`, 타임아웃이면 `fail`. ③ `-std=c++17 -O0 -g -Wall -m32` 에 `shims`와 스케치 디렉터리를 `include` 경로로 넣어 빌드. 컴파일 실패는 **명령줄**

`·stdout-stderr`를 전부 포함해 `fail`한다. ④ 실행(180초 타임아웃). **SIGSYS로 죽으면** — 일부 샌드박스가 32비트 ELF syscall 을 막는다 — `qemu-i386-static` 으로 같은 바이너리를 재실행한다. 성공 시 `[R1]` 로 시작하는 줄만 골라 출력한다.

10.4 check_repo_layout.py — 저장소 정합성 검사

```
def check_markdown_links(repo, files)
```

역할 — 유지 문서의 모든 상대 링크가 실제 파일을 가리키는지 확인.

기제 — 정규식으로 마크다운 링크를 뽑고, `<...>` 형식과 URL 인코딩을 풀고, 앵커(`#`)와 스킴이 있는 URL은 건너뛴다.

절대 경로 링크는 그 자체로 오류로 본다 (저장소를 옮기면 깨지기 때문). 나머지는 실제 파일 존재를 확인한다.

```
def check_sketch_names(repo)
```

역할 — 모든 `.ino` 의 디렉터리명과 파일명이 같은지 확인.

기제 — Arduino는 이 규칙을 강제하므로, 어기면 빌드가 실패한다. `archive/` 를 포함한 **전체** `firmware/` 를 검사한다.

```
def check_migration_map(repo, map_path, expected_rows=44)
```

역할 — 마이그레이션 맵 44행의 정합성 검증.

기제 — 네 가지를 확인한다. ① 행 수가 정확히 44, ② 옛 경로·새 경로에 중복 없음, ③ **옛 경로가 아직 존재하면 오류** (이동이 안 끝났다는 뜻), ④ **새 경로가 없으면 오류**(링크가 깨졌다는 뜻).

```
def check_stale_tokens(repo, files)
```

역할 — 폐기된 경로·파일명이 문서나 코드에 남아 있는지 검사.

기제 — `LEGACY_TOKENS` 13개 (`firmware/examples/` , `Drone_Control_Dualsense.py` , `ZETIN_Drone` 등)를 유지 문서 + `flight/·diagnostics/` 의 `.ino` + `scripts/*.py` 에서 찾는다. `migration_map.md` 는 정의상 옛 이름을 담으므로 **검사에서 제외**한다.

문서를 코드처럼 검사한다

일반적으로 문서의 정합성은 사람이 지키지만, 이 저장소는 **CI가 강제한다**. 링크가 깨지거나, 파일을 옮기고 문서를 안 고치거나, 옛 이름이 남으면 빌드가 실패한다. 26개 스케치와 수십 개 문서가 얹힌 구조에서 문서 부패를 막는 실질적인 장치다.

11. 문서와 로그: docs/, logs/

11.1 문서 체계

docs/ 는 유지 문서, 날짜가 박힌 기록, 보관 문서의 세 층으로 나뉜다. 핵심 규율은 "날짜 기록은 사후 수정하지 않고, 틀린 것으로 밝혀지면 정정을 병기한다"이다.

유지 문서 (현행 상태를 반영, 계속 갱신)

문서	내용
current_status.md	어디까지 했고 다음에 무엇을 하는가. 완료 기준 포함.
workflow.md	작업 순서와 인수인계 절차.
project_overview.md	전체 조감. 지원 범위·성숙도 표, 핀 배치, 좌표계, 믹서.
hardware_configuration.md	기체 구성과 실물로 확인할 항목.
firmware_catalog.md	26개 스케치 전체의 수명주기 표. 3.3절의 분류 근거.
udp_protocol.md (36KB)	명령·필드·바이너리 규격의 원본. 프로토콜 변경 시 여기가 기준.
power_on_bench_procedure.md (51KB)	첫 호버 전 안전 관문. 프로펠러 제거부터 단계별 절차와 실측 기록.
ground_station_link.md	링크 요구사항과 RC 타임아웃 진단. WiFi/BT 공유 라디오 문제의 근거.
bmm350_yaw_bench_test.md (14KB)	자기계 융합 벤치 4단계 시험 기록.
failsafe_land_research.md (29KB)	착지 감지 연구와 정정 기록. 5.1절의 철회 경위가 여기에 있다.
migration_map.md	옛 경로 → 새 경로 44행 대응표. CI가 검증한다.
notion_doc_convention.md	노션 문서 작성 규약.
cascade_vs_single_pid.typ / .pdf	단일 PID와 캐스케이드 PID 비교. Typst 원본과 생성 PDF를 함께 추적한다.

날짜 기록 (사후 수정 금지)

디렉터리	내용
superpowers/specs/	기능별 설계 6건 — Tier1 관측성, BMM350 yaw 융합, 자동착륙, yaw 각속도 명령, 자기계 캘리브레이션, 리뷰 수정.
superpowers/plans/	그 설계의 구현 계획 6건. 가장 큰 것은 2026-07-28-failsafe-vertical-state-strategy.md (53KB).

디렉터리	내용
design/ , plans/	2026-07-13 저장소 정리 설계와 실행 계획(45KB).
history/	2026-05-14 듀얼 IMU PID 설계와 구현 계획(47KB).
reports/	2026-09-07 문서·코드 정합성 점검 기록.
archive/	커밋 메시지 스냅샷, PID/DShot 발표 스니펫.

계획서를 완료 증거로 읽으면 안 된다

`firmware_catalog.md` 가 명시한다: "체크리스트나 예상 결과를 구현·실행 완료 증거로 사용하지 않는다." 실제로 8.2절의 `dual_imu_pid_pwm` 은 계획서에 있던 개별 IMU 폴백과 런타임 bias 갱신이 **구현되지 않은 채** 남았다. 계획과 구현의 괴리는 이 저장소에서 반복적으로 확인된 패턴이다.

11.2 로그 디렉터리

파일 패턴	내용
flight_log_YYYY-MM-DD_HHMMSS.csv	20Hz 텔레메트리 66열(타임스탬프 + 65필드). PC 백시계 기준.
imuraw_YYYY-MM-DD_HHMMSS.bin	1kHz IMU 원시 바이너리. ZIMU / ZCAL 레코드가 섞여 있다.
imuraw_*.csv	위 .bin 을 <code>decode_imu_raw.py</code> 로 변환한 것. 지상국 종료 시 자동 생성된다. 펌웨어 <code>micros()</code> 기준.

`logs/README.md` 만 Git에 추적되고 로그 자체는 대부분 `.gitignore` 대상이다. 두 로그의 **시간 기준이 다르다**는 점이 중요하며(4.3절), 그래서 ZIMU v2에 `failsafe_phase` 가 추가되었다.

12. 빌드·CI·설정 파일

12.1 GitHub Actions 워크플로

.github/workflows/ci.yml — 두 개의 잡

잡	단계
python	Python 3.12 설정 후 <code>requirements.txt</code> 를 그대로 설치한다. 주석에 이유가 있다: 개별 패키지를 골라 넣었더니 matplotlib 다음에 pandas가 또 걸렸다 — 목록이 갈라지지 않게 선언된 파일을 쓴다.
	32비트 툴체인 설치 — <code>gcc-multilib g++-multilib</code> .
	<code>python -m py_compile scripts/*.py</code> — 구문 검사.
	<code>python tools/check_repo_layout.py</code> — 레이아웃 검사.
	단위 테스트 — 인라인 Python으로 <code>unittest</code> 를 돌리고, <code>skip</code> 을 실패로 취급한다.
firmware	<code>~/arduino15</code> 캐시(키에 정확한 버전이 박혀 있다).
	ESP32 코어 <code>3.3.10</code> , <code>ICM42670P@1.0.9</code> , <code>ESP32Servo@3.2.1</code> , <code>DFRobot_BMM350</code> 은 Git URL로 설치(Arduino 라이브러리 인덱스에 없음).
	<code>flight/*</code> 와 <code>diagnostics/*</code> 를 <code>--warnings all</code> 로 컴파일. <code>set -e</code> 로 첫 실패에서 중단.

이 CI 파일에는 두 번의 "조용한 실패"가 주석으로 기록되어 있다

- `skip`이 CI를 녹색으로 만들었다. `multilib`가 없으면 네이티브 테스트가 예외 대신 `skipTest` 로 빠지고, `skip`은 실패가 아니므로 CI가 녹색인데 아무것도 검증하지 않았다. 2026-09-07 실행(34104011215)이 `failsafe land-attitude SIL-control math-BMM350 SIL-MSP` 센서-yaw 명령 등 7개를 건너뛰고 OK로 끝났다. 현재는 `result.skipped` 가 있으면 종료 코드 1을 반환하고 `::error::` 로 각 `skip`을 보고한다.
- 누락된 라이브러리가 한 달간 컴파일 검증을 없었다. 주력 스케치가 2026-07-24부터 `DFRobot_BMM350` 을 포함했는데 CI 설치 목록에 추가하지 않아 `firmware` 잡이 계속 실패했다 — 즉 약 한 달간 컴파일 검증이 없었다.

두 사례 모두 "테스트가 존재한다"와 "테스트가 실행된다"가 다르다는 것을 보여주며, 주석으로 남겨 재발을 막는다.

.github/workflows/pages-redirects.yml

GitHub Pages 리다이렉트 생성. 발표자료 사이트 (`lightminn.github.io/zetin-drone-presentations`)로 연결하는 보조 워크플로다.

12.2 의존성과 루트 설정 파일

파일	내용
<code>requirements.txt</code>	지상국 Python 의존성. CI와 로컬의 단일 진실. <code>pygame</code> , <code>numpy</code> , <code>pandas</code> , <code>matplotlib</code> , <code>pyserial</code> 계열.

파일	내용
<code>requirements - 복사본.txt</code>	정리 대상 <code>requirements.txt</code> 와 내용이 같은 285바이트 사본. 최근 커밋(<code>b3ec12b</code>) 메시지가 "삭제 후 재커밋 예정"이라고 밝히고 있다. 13.5절 참조.
<code>.gitignore</code>	로그, 빌드 산출물, Python 캐시 제외.
<code>.gitattributes</code>	줄바꿈·바이너리 취급 규칙.
<code>README.md</code>	저장소 진입점. 현재 상태 표, "여기서 시작" 안내, 빠른 확인 명령 , 안전 규칙 4개.

로컬 전체 검증 명령

```
# 저장소 루트에서 실행. 기체에 업로드하지 않는다.
python -m py_compile scripts/*.py
python tools/check_repo_layout.py
python -m unittest discover -s tools -p 'test_*.py' -v

# 주력·보류 후보·진단 스케치 9개
(
  set -e
  for sketch in firmware/flight/*/ firmware/diagnostics/*; do
    arduino-cli compile --warnings all --fqbn esp32:esp32:esp32s3 \
      --build-path "/tmp/zetin-$(basename "$sketch")" "$sketch"
  done
)
```

2026-09-07 로컬 검증 기준: **테스트 175개 · 스케치 컴파일 9개 통과**. 컴파일은 ESP32 코어 3.3.7 기준이며 CI 고정값 3.3.10의 실행 결과와 구분한다.

13. 부록

13.1 텔레메트리 65필드 전체 명세

PC 수신기는 맨 앞에 `Timestamp` 를 추가하므로 CSV는 66열이다. 필드는 **append-only**이며 중간 삽입이나 순서 변경은 모든 구버전 파서를 깨뜨린다. 원본 규격은 `docs/udp_protocol.md` .

#	필드명	타입	의미
1-3	Roll, Pitch, Yaw	float	추정 자세(deg). 기체축.
4-6	Gyro_X/Y/Z	float	융합 각속도(dps). 기체축, LPF 후.
7-9	Accel_X/Y/Z	float	융합 가속도(g). 기체축.
10	Throttle	int	현재 <code>base_throttle</code> (μ s).
11	Fault_RC	int	RC 타임아웃 래치.
12	Fault_Critical	int	<code>active_imus==0</code> 과도기울기 <code>!calibration_ok</code> 의 합성.
13-14	RC_Total_Pkts, RC_Dropped_Pkts	int	RC 수신 통계. resume 확인에 사용.
15-17	Fault_IMU1, Fault_IMU2, Fault_Disagree	int	IMU 고장 래치.
18	Active_IMUs	int	현재 사용 중인 IMU 수(0/1/2).
19	Mixer_Scaled	int	자세 명령이 실제로 축소됐는가.
20	Fault_Attitude	int	과도 기울기 래치.
21	Calibration_OK	int	부팅 캘리브레이션 성공 여부.
22	Armed	int	<code>!safety_lock</code> . 시동 거부 감지의 유일한 수단.
23-26	Motor_M1/M2/M3/M4	int	모터 출력(μ s). FL/RR/FR/RL.
27	PID_Loop_Hz	int	측정된 제어 루프 주파수.
28-30	TgtRate_Roll/Pitch/Yaw	float	바깥 루프가 낸 목표 각속도(dps).
31	MagHeading	float	틸트 보상 자기 방위각. 융합 OFF에서도 마지막 값 유지 → 활성 판단 불가.
32-34	Mag_X/Y/Z	float	자기장 XYZ. 도메인은 필드 65가 결정한다.
35	Yaw_Hold	int	heading 잠금 활성 여부.

#	필드명	타입	의미
36	Failsafe_Phase	int	0 NONE / 1 DESCENDING / 2 CUT_LANDED / 3 CUT_TIMEOUT / 4 CUT_ABORT.
37–38	Trim_Roll, Trim_Pitch	float	드론이 적용 중인 트림(deg).
39–40	Hover_Est, Hover_Valid	float/int	호버 스톱틀 추정(μ s)과 유효 플래그.
41	Failsafe_Probe_State	int	0 WAIT / 1 DIP / 2 EVALUATE / 3 UNAVAILABLE / 4 BLOCKED.
42	Failsafe_Probe_NoResponse	int	연속 무반응 횟수.
43	Failsafe_Probe_Response_G	float	최근 차분 응답(g). <code>pre_dip - dip_min</code> .
44–49	IMU1_Gyro_X/Y/Z, IMU1_Accel_X/Y/Z	float	IMU1 개별값. 기체축. 자이로는 LPF 후, 가속도는 LPF 없음.
50–55	IMU2_Gyro_X/Y/Z, IMU2_Accel_X/Y/Z	float	IMU2 개별값.
56–58	TgtAngle_Roll/Pitch/Yaw	float	목표 자세(deg). 트림이 이미 포함되어 있다.
59	Mag_Enabled	int	자기계 융합 활성화 여부의 유일한 진실.
60	Range_MM	int	거리계 원값. 범위 밖이면 음수이며 가까울 때와 멀 때가 같은 음수.
61	Range_Quality	int	0–255, 또는 -1 = 신선한 프레임 없음.
62–63	Flow_X, Flow_Y	int	광류 이동량.
64	Flow_Quality	int	0–255, 또는 -1.
65	Mag_Cal_Active	int	1이면 필드 32–34가 보정 전 raw μ T다.

바이너리 데이터그램

매직	크기	주기	내용
ZIMU	20 + 27n B (n ≤ 50)	약 20Hz 배치 (1kHz 샘플)	헤더(version, n_samples, batch_seq, base_t_us, dropped) + 샘플 (dt_us, IMU1-IMU2 gyro/accel raw, failsafe_phase).
ZCAL	60 B	1Hz	gyro_bias1/2, accel_scale1/2, gyro_scale, accel_scale, imu2_sign. 이것 없이는 raw를 물리량으로 복원할 수 없다.
GAINS,	가변 ASCII	요청 시	12개 게인 회신.

13.2 UDP 명령어 전체 목록

드론 기본 주소 `192.168.4.1:4210` . 모든 명령은 ASCII 문자열이다.

명령	인자	동작과 제약
<code>rcr <seq> <r> <p> <y></code>	uint32, deg, deg, dps	주력 조종 명령. yaw는 각속도. 20Hz 전송. 하강 중에는 위치독만 갱신.
<code>rc <seq> <r> <p> <y></code>	uint32, deg ×3	yaw가 절대 각도. 2.3토큰 벤치 형식도 허용.
<code>start</code>	—	시동. 7개 안전 게이트를 통과해야 한다(4.11절).
<code>stop</code>	—	즉시 해제. 무조건 수락.
<code>resume</code>	—	자동착륙 복구. 6개 거부 조건 검사.
<code>th <v></code>	1000–1900	collective 설정. 제어 창을 ±150μs로 재설정.
<code>trim <r> <p></code>	±10 deg	절대값 트림. 하강 중 거부.
<code>yaw <0 1></code>	0 또는 1	heading 잠금 오버라이드. 1은 무장 중 거부.
<code>mag <0 1></code>	0 또는 1	자기계 융합. 미초기화 상태에서 무장 중이면 거부.
<code>magcal <0 1></code>	0 또는 1	보정 데이터 수집. 1은 무장 중 거부.
<code>magc <x> <y> <z></code>	μT/μs ×3	throttle 간섭 계수. 0이면 비활성.
<code>raw <0 1></code>	0 또는 1	ZIMU 스트림. ON 전환 시 링버퍼를 비운다.
<code>gains</code>	—	12개 게인 회신 요청.
게인 설정 (2글자 + 값, 0–100, 하강 중 거부)		
<code>rp / ri / rd</code>	float	roll-pitch 각속도 P / I / D (동시 설정).
<code>ap · ar · at · ay</code>	float	각도 P — roll-pitch 동시 / roll / pitch / yaw.
<code>yp / yi / yd</code>	float	yaw 각속도 P / I / D.
<code>pa / ia / da</code>	float	roll-pitch 각속도 P/I/D (<code>tune_pid.py</code> 호환).
<code>pr / ir / dr</code>	float	roll 개별 P/I/D.
<code>pp / ip / dp</code>	float	pitch 개별 P/I/D.
<code>py / iy / dy</code>	float	yaw 개별 P/I/D.

13.3 안전 규칙 요약

반드시 지켜야 할 4가지

1. **모터 벤치와 펌웨어 업로드는 프로펠러를 제거한 상태에서 한다.** 전원 극성·핀·모터 번호·회전 방향·보정 부호·정지 경로를 먼저 확인한다. 단일 모터 식별은 `motor_id_single` 로 시작하고, 부팅 후 자동 구동하는 `motor_pwm_bench` 와 구분한다.
2. **게임패드는 USB로, 드론에 UDP를 보내는 프로그램은 하나만 사용한다.** 과거 WiFi/BT 공유 라디오 장비에서 블루투스 사용과 RC 정체가 함께 관측됐다.
3. **RC 단절이 안전한 착지를 보장하지 않는다.** 호버 추정·스로틀 조건에 따라 즉시 컷하거나 시간 기반으로 하강한다. 하강 중에는 접지 후에도 모터가 돌 수 있고, 시간이 끝날 때 아직 공중일 수 있다.
4. **명령 전송과 실제 무장·정지를 구분한다.** 정지만 요청할 때는 stdin `stop` 을 쓴다. X 버튼은 로컬 무장 추정에 따른 토글이며, 링크 복귀만으로 조종이 재개되지 않는다.

권장 확인 순서

1. 프로펠러를 제거한다.
2. `motor_id_single` 로 한 모터씩 번호·방향·PWM 반응을 확인한다.
3. `icm42670_single_raw` 과 `icm42670_dual_raw` 로 센서축과 두 IMU 부호를 확인한다.
4. `icm42670_dual_loop_debug` 로 추정각·루프 주기를 확인한다. 이 진단에는 PID·믹서가 없으므로 보정 방향은 여기서 확인할 수 없다.
5. 전원 극성, 비상 정지, RC timeout 자동착륙, 과도 기울기 정지를 확인한 **뒤에만** 제한된 테스트 리그에서 비행 후 보를 평가한다.

13.4 주요 함수 색인

함수	위치	절
제어 루프 핵심		
<code>pid_task</code>	<code>dual_imu_cascade_pwm.ino:1194</code>	4.9
<code>udp_task</code>	<code>dual_imu_cascade_pwm.ino:1997</code>	4.10
<code>mixAndDesaturate</code>	<code>dual_imu_cascade_pwm.ino:1037</code>	4.7
<code>compute_alpha</code>	<code>dual_imu_cascade_pwm.ino:1177</code>	4.4
<code>checkFreeze</code>	<code>dual_imu_cascade_pwm.ino:562</code>	4.4
<code>rcTimedOut</code>	<code>dual_imu_cascade_pwm.ino:598</code>	4.4
<code>writeMotor</code> / <code>stopMotors</code>	<code>dual_imu_cascade_pwm.ino:1025</code>	4.7
안전·자동착륙		
<code>applyPendingSafetyRequest</code>	<code>dual_imu_cascade_pwm.ino:752</code>	4.5

함수	위치	절
<code>resumeRefusalReason</code>	<code>dual_imu_cascade_pwm.ino:787</code>	4.5
<code>updateHoverThrottleEstimator</code>	<code>failsafe_land.h:36</code>	5.1
<code>failsafeProbeDipUs</code>	<code>failsafe_land.h:81</code>	5.1
<code>updateLandDetector</code>	<code>failsafe_land.h:166</code>	5.1
<code>recordLandDetectorProbeDelivery</code>	<code>failsafe_land.h:147</code>	5.1
<code>failsafeStep</code>	<code>failsafe_land.h:248</code>	5.1
자세-yaw		
<code>updateYawOuter</code>	<code>yaw_command.h:15</code>	5.2
<code>yawTargetRateDps</code>	<code>yaw_command.h:30</code>	5.2
<code>tiltCompensatedMagHeadingDeg</code>	<code>mag_yaw_fusion.h:19</code>	5.3
<code>magYawCorrectionDeg</code> / <code>wrapDeg</code>	<code>mag_yaw_fusion.h:13, 38</code>	5.3
<code>applyMagCalibration</code>	<code>dual_imu_cascade_pwm.ino:979</code>	4.6
파서-통신		
<code>handleRcCommand</code> / <code>handleRcrCommand</code>	<code>dual_imu_cascade_pwm.ino:1735, 1782</code>	4.11
<code>setRcTargets</code>	<code>dual_imu_cascade_pwm.ino:1719</code>	4.11
<code>parseFloatStrict</code> / <code>parseIntStrict</code>	<code>dual_imu_cascade_pwm.ino:1699, 1709</code>	4.11
<code>sendTelemetry</code>	<code>dual_imu_cascade_pwm.ino:1861</code>	4.12
<code>sendImuRawBatch</code> / <code>sendImuCalibration</code>	<code>dual_imu_cascade_pwm.ino:1907, 1942</code>	4.12
<code>mSPSensorParseByte</code>	<code>mSP_sensor.h:101</code>	5.4
지상국		
<code>parse_telemetry_packet</code>	<code>scripts/telemetry_schema.py:221</code>	9.2
<code>telemetry_thread</code>	<code>scripts/control_dualsense.py:692</code>	9.3
<code>controller_thread</code>	<code>scripts/control_dualsense.py:902</code>	9.3
<code>advance_resume_attempt</code>	<code>scripts/control_dualsense.py:296</code>	9.3
<code>disarm</code> / <code>stop_streaming_only</code>	<code>scripts/control_dualsense.py:172, 200</code>	9.3

함수	위치	절
<code>_parse_capture</code>	scripts/decode_imu_raw.py:139	9.5
<code>fit_calibration</code>	scripts/magcal_fit.py:280	9.6
<code>_fit_unit_ellipsoid</code>	scripts/magcal_fit.py:191	9.6
<code>collect_failsafe_analysis</code>	scripts/analyze_flight_log.py:159	9.7
검증		
<code>check_migration_map</code> / <code>check_stale_tokens</code>	tools/check_repo_layout.py:73, 104	10.4

13.5 코드 분석에서 발견한 구조적 관찰과 개선 제안

이 절은 원본 요청의 "더 필요하다고 판단되는 내용"에 해당한다. 코드를 전부 읽는 과정에서 관찰한 강점과, 실제로 조치 가능한 개선 후보를 정리했다. 우선순위와 근거를 함께 적었으며, 기능 추가보다 이미 있는 것의 적합성을 높이는 쪽에 무게를 두었다.

A. 이 코드베이스의 구조적 강점

강점	구체적 근거
상수에 측정 근거가 붙어 있다	<code>Ki=0.05</code> (SIL 측정), <code>FS_MAX_MS=3000</code> (3회 실측 역산), <code>MAG_HARD_IRON</code> (1255샘플 중 inlier 1119, 교차검증 0.29μT, 물리 적합성 0.6%). 값만 있고 근거가 없는 상수가 거의 없다.
기각된 대안도 기록한다	SIL 완전 트림 값 0.5를 "실기 지연·노이즈에서 진동 위험"으로 의도적으로 피했다는 기록, 프로브 딥 2배 시도가 전달률 계산으로 기각된 기록. 나중에 같은 시도를 반복하지 않게 한다.
틀린 분석을 철회하고 그 사실을 남긴다	"공중 CUT_LANDED" 관측이 20Hz 단일 샘플 오독이었음을 밝히고 <code>07231eb</code> 에서 철회. 그 위에 세운 결론 4개도 함께 무효화했다. 소프트웨어 프로젝트에서 보기 드문 규율이다.
순서 제약을 주석으로 못 박는다	"이 블록은 RC timeout 가드보다 먼저 실행해야 한다", "가드가 없으면 Serial이 TX 버퍼를 포화시켜 워치독 재부팅" 등, 리팩터링이 깨뜨릴 수 있는 암묵적 불변식 을 명시했다.
컴파일 타임 불변식	7개 <code>static_assert</code> 가 안전 상수 간 관계를 강제한다. 특히 프로브 시간 배치 어서션은 변조 검사 (mutation test)로 오차를 발견해 고친 것 이다.
문서를 CI가 검사한다	링크 유효성, 스케치 명명, 마이그레이션 맵 44행, 폐기 토큰 잔존을 자동 검증한다.
실패한 CI를 주석으로 남긴다	<code>skip</code> 이 녹색을 만든 사건, 라이브러리 누락으로 한 달간 컴파일 검증이 없었던 사건을 워크플로 파일에 기록했다.

B. 즉시 조치 가능한 항목

우선	항목	근거와 제안
높음	requirements - 복사본.txt 삭제	requirements.txt와 바이트 단위로 동일함을 확인했다. 최근 커밋 b3ec12b의 메시지도 "삭제 후 재커밋 예정"이라고 밝힌다. 의존성 목록이 둘로 갈라지면 CI와 로컬이 어긋나는데, 이는 CI 주석이 이미 경계한 문제다. 공백이 포함된 한글 파일명이라 셸 스크립트에서도 취급이 번거롭다.
높음	check_repo_layout.py에 루트 파일 검사 추가	현재 검사기는 문서 링크·스케치명·마이그레이션 맵·폐기 토큰을 본다. 위와 같은 중복 사본 파일은 잡지 못한다. "복사본", "copy", "_bak" 같은 패턴을 LEGACY_TOKENS와 유사한 방식으로 파일명 검사에 추가하면 재발을 막는다.
중간	pwm_RGB.cpp 0바이트 파일	archive/platformio_skeleton/src/의 빈 파일이다. 보관 구역은 원형 보존이 원칙이므로 삭제 대신, platformio_skeleton/README.md에 "착수만 되고 구현되지 않은 파일"임을 한 줄 명시하는 편이 규율에 부합한다.
중간	두 펌웨어의 yaw 부호 차이를 빌드 시점에 노출	주력은 CW+, 보류 후보는 CCW+이고 계인 단위도 다르다. 현재는 README와 주석에만 적혀 있다. 보류 후보 쪽에 #warning이나 부팅 시리얼 배너에 "NOT COMPATIBLE WITH control_dualsense.py (rc only, CCW+ yaw)"를 넣으면, 잘못된 펌웨어를 올린 상태에서 조종을 시도하는 사고를 한 단계 앞에서 막는다.
중간	프로브 관련 죽은 코드의 처리 방침 명시	updateLandDetector의 반환값은 (void)로 버려지고 landed는 false 고정이다. 이는 의도된 상태이며 주석도 충실하다. 다만 FS_PROBE_RESPONSE_G · FS_PROBE_CONFIRM_N처럼 "판정에 쓰이지 않는다"고 명시된 상수가 여럿 남아 있다. 이들을 // DIAGNOSTIC-ONLY 같은 일관된 마커로 묶으면, 나중에 거리계 기반 판정을 도입할 때 교체 대상을 한 번에 식별할 수 있다.
낮음	네이티브 테스트 실행 시간	SIL 타임아웃이 180초다. 케이스가 늘면 CI 시간이 선형으로 증가한다. 회귀 계열(R1, V1-V6)과 단위 계열을 분리해 PR에서는 후자만, main 병합에서는 전체를 돌리는 구성을 검토할 수 있다.

C. 설계 수준의 관찰 (조치 전 논의 필요)

관찰	내용
가장 큰 미해결 위험은 착지 감지다	자동착륙은 고도나 접지를 전혀 확인하지 않고 3초간 내려간 뒤 자른다. 접지 후에도 프로펠러가 돌고, 시간이 끝날 때 공중일 수도 있다. IMU만으로 이를 푸는 시도는 "교란만으로 프로브 신호 전체 크기가 나온다" (p95 0.0904g vs 임계 0.06g)는 측정으로 한계가 증명되었다. 3901-L0X 거리계가 이미 장착되어 데이터가 텔레메트리로 나오고 있으므로, 다음 작업의 자연스러운 방향은 거리계를 제어 페루프에 넣는 것이다. 다만 거리 원값의 부호 모호성(5.4절)과 품질 판정 규칙을 먼저 확립해야 한다.
hover_valid가 만료되지 않는 것의 양면성	"낮은 추정치가 덜 위험하다"는 판단은 타당하다. 다만 기체 질량이 비행 중 바뀌거나(페이로드 투하) 배터리 전압이 크게 떨어지면 추정치가 실제 호버에서 멀어진다. 추정치의 나이를 텔레메트리에 노출하면 (현재는 Hover_Valid 불리언만 있다) 지상국이 판단할 여지가 생긴다.
텔레메트리가 원자적 스냅샷이 아니다	파서 docstring이 명시하듯 필드가 서로 다른 tick에서 올 수 있다. 현재 분석 도구들은 이를 알고 작성되었으나, 새 분석 코드를 쓰는 사람이 이 사실을 놓치기 쉽다. 20Hz CSV에 PID_Loop_Hz 같은 tick 식별자를 추가하는 것은 append-only 규칙을 지키면서 가능한 보강이다.

관찰	내용
단일 .ino 2,360행	Arduino 전처리기 제약 때문에 분할이 제한되는 것은 사실이다. 다만 명령 파서군(:1691-1851)과 텔레메트리 송신군 (:1861-1995)은 순수 함수에 가까워 추가 헤더로 분리할 수 있고 , 그렇게 하면 네이티브 테스트에서 .ino 전체를 컴파일하지 않고도 검사할 수 있다.

총평

21,000행 규모의 취미·연구 성격 드론 프로젝트로서는 **이례적으로 높은 수준의 공학적 규율**을 갖추고 있다. 특히 (1) 상수마다 측정 근거와 기각된 대안을 남기는 습관, (2) 틀린 분석을 철회하고 그 경위를 코드에 기록하는 규율, (3) 실제 .ino 를 호스트에서 페루프로 돌리는 SIL 검증, (4) 문서 정합성을 CI가 강제하는 구조는 상용 제품 코드에서도 자주 보기 어렵다.

반대로 가장 큰 기술적 공백은 명확하다: **고도와 접지를 측정하는 센서가 제어 페루프에 들어가 있지 않다**. 이 때문에 자세 제어는 검증되었음에도 자동 호버·자동 착륙이 성립하지 않는다. 센서는 이미 기체에 있고 데이터도 흐르고 있으므로, 다음 단계는 새 하드웨어가 아니라 **이미 있는 관측을 제어에 연결하는 일**이다.